

URIB Rail Integration Manual

Universal Bridge (URIB) — Rail Adapter Architecture & Integration Reference

Abstract

The Universal Rail Integration Bridge (URIB) is a deterministic settlement infrastructure designed to normalise heterogeneous payment rails — including Bitcoin, XRP Ledger, ISO 20022 messaging networks, Central Bank Digital Currency (CBDC) platforms, EVM-compatible blockchains, Stellar, and emerging custom rails — into a single canonical semantic representation known as the **Canonical Semantic Thread (CST)**. By abstracting rail-specific semantics behind a rigorously specified adapter interface (the Rail Adapter Interface, or RAI), URIB enables atomic cross-rail settlement equivalence: any obligation originating on one rail can be provably settled, disputed, or voided with respect to obligations on any other rail, without either party requiring knowledge of the counterparty's underlying rail architecture. Identity continuity across rails is maintained through the **Identity Continuity Token (ICT)** primitive, ensuring that participants are represented consistently regardless of the rail on which they operate. This manual constitutes the authoritative engineering reference for teams implementing new rail adapters, integrating existing adapters, or operating URIB infrastructure.

Table of Contents

Front Matter

Glossary of Terms

Section 1 — URIB Architecture Overview

1.1 Design Philosophy

1.2 System Layers

1.3 Architecture Diagram Description

1.4 Bridge Epoch Model

Section 2 — Rail Adapter Interface (RAI) Specification

2.1 RAI Contract Definition

2.2 RailNamespace Registry

2.3 CanonicalAmount (C_AMT) Specification

2.4 CanonicalSemanticThread (CST) Schema

2.5 FinalityVector Specification

Section 3 — Rail Invariants

3.1 Universal Rail Invariants

3.2 Rail-Family Invariants

3.3 Invariant Validation Pipeline

3.4 Invariant Violation Handling

Section 4 — Canonical Semantic Thread Mapping Rules

4.1 Mapping Philosophy

4.2 Mapping Rule Set (CST-M-001 through CST-M-006)

4.3 Mapping Error Codes

Section 5 — Settlement Equivalence Logic

5.1 Settlement Equivalence Proof (SEP) Overview

5.2 SEP Data Structure

5.3 SEP Generation Algorithm

5.4 Cross-Rail Atomic Settlement

5.5 Settlement Equivalence Rules Table

5.6 Dispute Resolution Protocol

Section 6 — Identity Continuity Layer (ICL)

6.1 ICT Architecture

6.2 Identity Mapping Registry

6.3 Identity Mapping Templates (BTC, XRP, ISO20022, CBDC, EVM, Stellar)

6.4 Cross-Rail Identity Resolution

6.5 Key Rotation and ICT Migration

Section 7 — Per-Rail Adapter Specifications

7.1 Bitcoin (BTC) Rail Adapter

7.2 XRP Ledger Rail Adapter

7.3 ISO 20022 Rail Adapter

7.4 CBDC Rail Adapter

7.5 EVM Rail Adapter (Ethereum & L2s)

7.6 Stellar Rail Adapter

7.7 Generic Rail Adapter Template

Section 8 — Worked Examples

8.1 Bitcoin → Ethereum ERC-20 Atomic Swap

8.2 XRP → SEPA ISO 20022 Payment

8.3 CBDC Retail Transfer with Compliance Assertion

8.4 Stellar Path Payment Bridged to Lightning Network

8.5 Multi-Rail Obligation Graph Settlement

Section 9 — Bridge Orchestrator & Epoch Management

9.1 Epoch Lifecycle Management

9.2 Obligation Graph

9.3 Validator Set & Threshold Signatures

9.4 Dispute Resolution Engine

9.5 Monitoring & Alerting

Section 10 — Security Considerations

10.1 Threat Model

10.2 Rail Adapter Sandboxing

10.3 Replay Attack Prevention

10.4 Economic Security

10.5 Cryptographic Agility

Section 11 — Deployment & Operations

11.1 Adapter Registration Process

11.2 Configuration Reference

11.3 Testing Requirements

11.4 Upgrade & Versioning

Appendices

Appendix A — Math Primitives Reference

Appendix B — RailNamespace Registry (Current)

Appendix C — ICT Derivation Reference

Appendix D — Error Code Reference

Appendix E — Changelog

Glossary of Terms

URIB (Universal Rail Integration Bridge)

The overarching infrastructure system that normalises heterogeneous payment and settlement rails into a single canonical semantic framework, enabling atomic cross-rail settlement equivalence and identity continuity.

Rail Adapter

A software component implementing the Rail Adapter Interface (RAI) that translates rail-native transaction semantics into URIB Canonical Semantic Threads. Each rail family has a dedicated adapter.

Canonical Semantic Thread (CST)

The normalised, rail-agnostic representation of a payment or settlement event. A CST carries all information necessary to verify, settle, and audit a transaction without reference to its originating rail's native data formats.

Settlement Equivalence Proof (SEP)

A cryptographic attestation, produced by the Settlement Equivalence Engine, that a set of CSTs across two or more rails collectively satisfies a settlement obligation. The SEP is the primary deliverable of the cross-rail settlement process.

Rail Invariant

A logical constraint that must hold for any valid CST or set of CSTs produced by a rail adapter. Universal invariants (INV-U-001 through INV-U-008) apply to all rails; rail-family invariants apply to specific rail families.

Math Primitive

A foundational cryptographic operation — including hash functions, commitment schemes, elliptic curve operations, and zero-knowledge gadgets — used as a building block throughout URIB. All math primitives are defined in Layer 0.

Identity Continuity Token (ICT)

A 32-byte canonical identifier that uniquely and persistently represents a participant across all rails within URIB. Derived deterministically from the participant's canonical public key and rail namespace.

Cross-Rail Commitment (CRC)

A commitment published to the CST Engine that binds a rail adapter's output to a specific CST and epoch anchor. Once published, a CRC is irrevocable by the adapter unilaterally (see INV-U-004).

Rail Namespace

A structured string identifier uniquely identifying a rail and network in the URIB registry. Format:

`<rail-family>[:<chain-id>][:<network>]`. Example: `evm:1:mainnet`.

Canonical Amount (C_AMT)

A 256-bit unsigned integer representing monetary value in URIB's internal base unit, defined as 10^{-18} of a reference SDR basket. All rail-native amounts are converted to C_AMT before entering the CST Engine.

Finality Vector

A structured descriptor attached to every CST that captures the confirmation state of the underlying rail transaction, including confirmation count, finality type, and an `isFinal` boolean flag.

Rail Anchor

A reference — typically a transaction ID, block hash, or ledger index — that anchors a URIB commitment or settlement event to a specific, verifiable location on the originating rail's ledger or chain.

Commitment Digest

A 32-byte hash value committed in the CST that binds together the thread's canonical data (amount, identities, obligation, semantic payload) in a tamper-evident manner. Used during SEP verification.

Bridge Epoch

A discrete, time-bounded settlement window (default: 600 seconds) during which CSTs are accumulated, validated, and settled. All cross-rail commitments must reference the epoch's anchor hash.

Obligation Graph

A directed acyclic graph (DAG) maintained by the Bridge Orchestrator where each node represents a settlement obligation and edges represent CST dependencies. Used to determine settlement ordering via topological sort.

Section 1 — URIB Architecture Overview

1.1 Design Philosophy

URIB is designed around four non-negotiable engineering principles that govern every layer of the system.

Single canonical truth across all rails. Every payment event, regardless of its rail of origin, must be expressible as a Canonical Semantic Thread. The CST is the single source of truth for settlement decisions. No settlement logic may operate on rail-native data; all computation occurs on canonical representations.

Composability without rail knowledge leakage. A component operating on a CST must never require knowledge of the originating rail's semantics, data formats, or consensus rules. Rail specifics are fully encapsulated within the adapter layer. This property enables the Settlement Equivalence Engine and Identity Continuity Layer to be rail-agnostic by construction.

Deterministic settlement proofs regardless of rail semantics. A SEP for a given set of CSTs must be computable deterministically by any conformant validator, producing identical outputs for identical inputs. Non-determinism in settlement proof generation is a critical defect. All sources of non-determinism — including FX rate lookups and finality assessments — are pinned at epoch-open.

Identity continuity as a first-class primitive. Rail adapters must not treat identity mapping as an afterthought. The ICT must be computed and bound to every CST before the thread is admitted to the CST Engine. An unresolvable identity is a hard rejection condition, not a soft warning.

1.2 System Layers

URIB is organised into six horizontal layers. Each layer has a strictly defined set of responsibilities and communicates with adjacent layers through typed interfaces only.

Layer 0 — Math Primitives

Layer 0 defines the cryptographic building blocks used throughout URIB. It is not a runtime component but a specification layer: all permissible hash functions (SHA256, SHA256d, SHA512/256, Keccak256), commitment schemes (Pedersen, hash-based), elliptic curve groups (secp256k1, Ed25519, BLS12-381), and zero-knowledge gadgets (hash preimage proofs, range proofs) are defined here. Every higher layer must reference only Math Primitives defined in Layer 0. Upgrading a primitive requires a versioned migration procedure (see Section 10.5).

Layer 1 — Rail Adapter Interface (RAI)

Layer 1 defines the **Rail Adapter Interface** — the standard contract that every rail adapter must implement. The RAI specifies methods for canonical normalisation of amounts, identities, and transactions; commitment construction and verification; settlement submission; and invariant validation. Rail adapters are the only components permitted to communicate with external rail nodes. No higher layer interacts directly with any rail network.

Layer 2 — Canonical Semantic Thread (CST) Engine

Layer 2 is the normalisation and threading engine. It receives Cross-Rail Commitments from Layer 1 adapters, validates them against universal invariants (Section 3), assigns them to the active epoch, and maintains the global threadId registry. The CST Engine emits events consumed by Layers 3, 4, and 5. It is the central, authoritative store of all in-flight and settled CSTs.

Layer 3 — Settlement Equivalence Engine (SEE)

Layer 3 consumes CSTs from the CST Engine and constructs Settlement Equivalence Proofs. It implements the SEP generation algorithm (Section 5.3), the two-phase commit protocol for ATOMIC obligations (Section 5.4), and the conservation verification logic. The SEE queries the ICL (Layer 4) for identity resolution and receives epoch management signals from the Bridge Orchestrator (Layer 5).

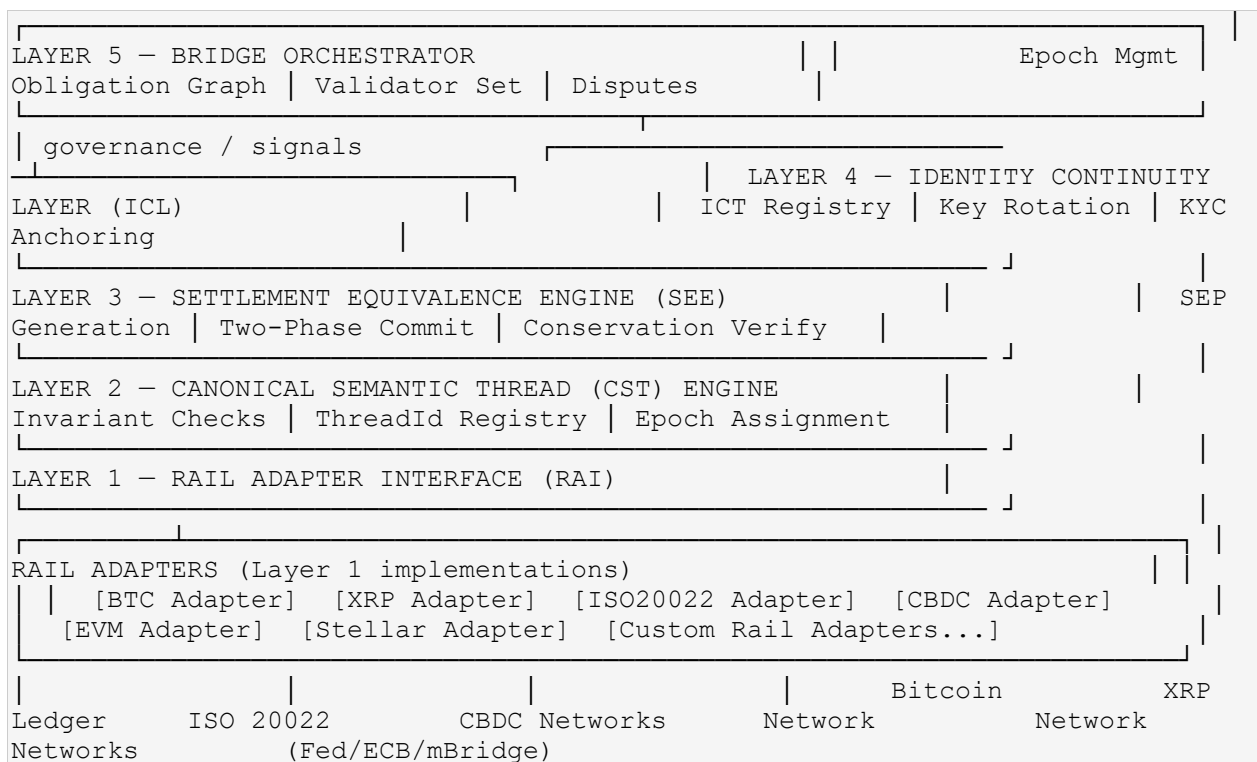
Layer 4 — Identity Continuity Layer (ICL)

Layer 4 maintains the Identity Mapping Registry and provides ICT resolution services to all other layers. It enforces identity binding invariants (INV-U-002), manages key rotation and ICT versioning, and processes KYC/AML attestation anchoring for CBDC rails. The ICL is a read-heavy component; writes occur only during registration or key rotation events.

Layer 5 — Bridge Orchestrator

Layer 5 is the apex governance component. It manages epoch lifecycle transitions, maintains the Obligation Graph, coordinates the validator set for threshold signature production, and operates the dispute resolution engine. The Bridge Orchestrator does not perform any cryptographic computation on transaction data; it orchestrates the other layers and enforces system-level policies.

1.3 Architecture Diagram



Data flows upward: rail nodes are queried by adapters; adapters push CRCs to the CST Engine; the CST Engine feeds the SEE and ICL; the SEE produces SEPs; the Bridge Orchestrator governs all state transitions. No downward data flow occurs outside of settlement submission, which is initiated by an adapter under Orchestrator instruction.

1.4 Bridge Epoch Model

Section 2 — Rail Adapter Interface (RAI)

Specification2.1 RAI Contract Definition

Every rail adapter must fully implement the following interface. Partial implementations are not permitted; missing method implementations cause adapter registration to fail at the URIB Review Board audit phase. All type names referenced here are defined in the URIB Core Type Registry.

```
interface RailAdapter {    // — Identity
    readonly
    railNamespace: RailNamespace;    // Canonical namespace string, e.g.
    "btc:mainnet", "evm:1:mainnet"    readonly railVersion: semver;    // Adapter
    software version. Must match registered version in RailNamespace Registry.
    // — Math Primitives
    hashFunction(): HashAlgorithm;    // Returns the hash algorithm used for
    transaction hashing on this rail.    // Must be one of: SHA256 | SHA256d |
    SHA512_256 | Keccak256    commitmentScheme(): CommitmentScheme;    // Returns
    the commitment scheme used for CrossRailCommitments.    // Must be one of:
    PedersenCommitment | HashCommitment    signatureScheme(): SignatureScheme;
    // Returns the signature scheme for identity binding proofs.    // Must be one
    of: ECDSA_secp256k1 | Ed25519 | Schnorr_secp256k1 | BLS12_381    // —
    Canonical Normalisation
    normaliseAmount(nativeAmt: bigint, nativeDenom: string): CanonicalAmount;
    // Converts a rail-native amount to C_AMT.    // MUST use floor division. MUST
    NOT accept negative amounts.    // Throws E-MAP-002 on overflow.
    normaliseIdentity(nativeAddr: string): IdentityContinuityToken;    // Resolves
    a rail-native address to an ICT via the ICL Identity Mapping Registry.    //
    Throws E-MAP-003 if address cannot be resolved and provisional ICT cannot be
    issued.    normaliseTransaction(nativeTx: RawTransaction):
    CanonicalSemanticThread;    // Full normalisation: returns a complete,
    validated CST.    // This is the primary entry point for transaction
    ingestion.    // MUST be a pure function: idempotent, no side effects.    //
    — Commitment Interface
    buildCommitment(cst: CanonicalSemanticThread): CrossRailCommitment;    //
    Constructs a CRC from a validated CST. Does NOT publish the commitment.
    verifyCommitment(crc: CrossRailCommitment, proof: bytes): boolean;    //
    Verifies that a CRC is validly constructed and the proof is correct.    //
    — Settlement Interface
    submitSettlement(sep: SettlementEquivalenceProof): RailAnchor;    // Submits
    the settlement instruction to the rail (e.g., publishes OP_RETURN,    // emits
    contract event, inserts Memo field). Returns the RailAnchor.
    queryFinality(railAnchor: RailAnchor): FinalityVector;    // Queries the rail
    node for the current finality state of the given anchor.    // Must be
    callable multiple times; each call returns current state.    // — Rail
    Invariant Checks
    validateInvariants(cst: CanonicalSemanticThread): InvariantResult;    // Runs
    the full three-phase validation pipeline (structural, semantic, rail-
    specific).    // Returns InvariantResult { passed: boolean; violations:
    InvariantViolation[] } }
```

2.2 RailNamespace Registry

The **Rail Namespace** is a structured string that uniquely identifies a rail and network within URIB. The format is: `<rail-family>[:<chain-id>][:<network>]`. The `rail-family` segment is a short lowercase ASCII identifier. The `chain-id` segment is used for EVM-compatible chains. The `network` segment distinguishes mainnet from testnet deployments.

Namespace	Description	Hash Function	Signature Scheme	Finality Model	Native Denomination
<code>btc:mainnet</code>	Bitcoin mainnet	SHA256d	Schnorr / ECDSA secp256k1	Probabilistic	satoshi (10^{-8} BTC)
<code>btc:testnet4</code>	Bitcoin Testnet 4	SHA256d	Schnorr / ECDSA secp256k1	Probabilistic	satoshi
<code>xrp:mainnet</code>	XRP Ledger mainnet	SHA512/256	ECDSA secp256k1 / Ed25519	Deterministic	drop (10^{-6} XRP)
<code>iso20022:sepa</code>	SEPA payment messaging	SHA256 (XML c14n)	RSA-PSS / ECDSA	Deterministic (instant)	EUR (2 decimal places)
<code>iso20022:swift</code>	SWIFT payment messaging	SHA256 (XML c14n)	RSA-PSS	Deterministic (T+0/T+1)	Multi-currency (ISO 4217)
<code>iso20022:fednow</code>	FedNow instant payments	SHA256 (XML c14n)	RSA-PSS	Deterministic (instant)	USD (2 decimal places)
<code>evm:1:mainnet</code>	Ethereum mainnet	Keccak256	ECDSA secp256k1	Probabilistic	wei (10^{-18} ETH)
<code>evm:137:polygon</code>	Polygon PoS mainnet	Keccak256	ECDSA secp256k1	Probabilistic (L2)	wei (10^{-18} MATIC/POL)
<code>evm:42161:arbitrum</code>	Arbitrum One mainnet	Keccak256	ECDSA secp256k1	Probabilistic (L2)	wei (10^{-18} ETH)

Namespace	Description	Hash Function	Signature Scheme	Finality Model	Native Denomination
evm:10:optimism	OP Mainnet	Keccak256	ECDSA secp256k1	Probabilistic (L2)	wei (10^{-18} ETH)
evm:8453:base	Base mainnet	Keccak256	ECDSA secp256k1	Probabilistic (L2)	wei (10^{-18} ETH)
stellar:mainnet	Stellar mainnet	SHA256 (XDR)	Ed25519	Deterministic	stroop (10^{-7} XLM)
cbdc:usd:fed	US Federal Reserve CBDC (hypothetical)	SHA256	ECDSA secp256k1	Deterministic	Platform-native (18 decimals)
cbdc:eur:ecb	ECB Digital Euro	SHA256	ECDSA secp256k1	Deterministic	Platform-native (18 decimals)
cbdc:multi:mbridge	Project mBridge multi-CBDC	SHA256 / Keccak256	ECDSA secp256k1	Deterministic	Platform-native (varies)

2.3 CanonicalAmount (C_AMT) Specification

The **Canonical Amount** (C_AMT) is a 256-bit unsigned integer representing monetary value in the URIB internal base unit. One URIB base unit equals 10^{-18} of a reference Special Drawing Rights (SDR) basket, as defined by the URIB FX Oracle specification. The SDR basket composition is fixed at epoch-open and not recalculated mid-epoch.

The canonical conversion formula is:

```
C_AMT = floor( nativeAmount × 10^(18 - nativeDecimals) × epochFxRate )
Where:  nativeAmount  - raw integer amount in the smallest rail-native unit (bigint)
        nativeDecimals - number of decimal places in the rail-native denomination
        epochFxRate    - rational number committed in the epoch anchor, expressed as (fxRateNumerator / fxRateDenominator), both uint256
        floor()         - truncation toward zero (mandatory; see INV-U-007)
```

Critical — Rounding Rule

Rounding MUST always be floor (toward zero). Any adapter that rounds up, rounds to nearest, or uses floating-point arithmetic at any intermediate step violates INV-U-007 and creates synthetic value. All intermediate calculations must use 256-bit unsigned integer arithmetic with explicit overflow checking.

The FX rate is provided by the URIB FX Oracle, which publishes a signed rate commitment at epoch-open. This commitment is part of the epoch anchor hash computation. Adapters that detect a stale FX rate (oracle commitment age exceeding `fx_staleness_threshold`) must throw `E-MAP-011` and refuse to normalise amounts until the epoch anchor is refreshed.

Rail	Native Unit	Decimal Places	Conversion Factor ($10^{(18-d)}$)	Example: 1 base unit $\rightarrow$ C_AMT
Bitcoin	satoshi	8	10^{10}	1 sat = $10,000,000,000 \times \text{fxRate}$
XRP	drop	6	10^{12}	1 drop = $1,000,000,000,000 \times \text{fxRate}$
Ethereum / EVM	wei	18	$10^0 = 1$	1 wei = $1 \times \text{fxRate}$
Stellar (XLM)	stroop	7	10^{11}	1 stroop = $100,000,000,000 \times \text{fxRate}$
ISO 20022 (EUR)	euro-cent	2	10^{16}	1 cent = $10^{16} \times \text{fxRate}$
CBDC (18-decimal)	platform unit	18	$10^0 = 1$	1 unit = $1 \times \text{fxRate}$

2.4 CanonicalSemanticThread (CST) Schema

The CST is the canonical, rail-agnostic representation of a payment or settlement event. It is the fundamental unit of value transfer within URIB. Every field is mandatory unless explicitly noted as optional.

```
interface CanonicalSemanticThread {    threadId:          bytes32;    // Globally unique thread identifier.    // Computed as: keccak256(abi.encode(railNamespace, nativeTxId, epochAnchor))    // No two CSTs may share a threadId across any rail or epoch (INV-U-006).    epochAnchor:      bytes32;    // The epoch root hash of the OPEN epoch at time of commitment.    // CSTs committed to a CLOSING or SETTLED epoch are rejected.    railNamespace:    string;    // Originating rail namespace, e.g. "btc:mainnet". Must match a registered namespace.    sourceIdentity:    ICT;    // Identity Continuity Token of the sending party.    // Must be fully resolved (not provisional) for TRANSFER, ESCROW, ATOMIC obligations.    // Provisional ICT permitted for BURN, MINT with governance authorisation.    destinationIdentity: ICT;    // ICT of the receiving party. Same resolution rules as sourceIdentity.    canonicalAmount:    C_AMT;    // 256-bit unsigned integer in URIB base units. Must be > 0.    canonicalFee:        C_AMT;    // 256-bit unsigned integer in URIB base units. May be 0 for fee-free rails.    // For EVM: includes baseFee + priorityFee converted to C_AMT.    // Conservation check: sourceDebit == canonicalAmount + canonicalFee.    obligation:          ObligationType;    // One of: TRANSFER | ESCROW | BURN | MINT | SWAP | ATOMIC    // Determined by CST-M-003 mapping rule.    semanticPayload:      bytes;    // ABI-encoded rail-specific metadata (see CST-M-005 for encoding per rail family).    // Must hash to a value committed within commitmentDigest (INV-U-008).    timestamp:            uint64;    // Unix epoch milliseconds at time of normalisation. Not the on-chain timestamp.    // On-chain timestamp is encoded in semanticPayload.    finalityVector:        FinalityVector;    // Current finality state. Must be refreshed before SETTLED promotion (CST-M-006).    commitmentDigest:     bytes32;    // Binds together: keccak256(abi.encode(    // threadId, epochAnchor, sourceIdentity, destinationIdentity,    // canonicalAmount, canonicalFee, obligation, keccak256(semanticPayload)    // ))    // Any modification to any committed field invalidates the commitmentDigest.    parentThreadId:        bytes32 | null;    // Optional. Set for chained threads (CST-M-004). Null for independent threads.    // Must reference a threadId that already exists in the CST Engine's registry. }
```

The **threadId** computation ensures global uniqueness because it combines three globally unique components: the rail namespace (unique per rail/network), the native transaction ID (unique per rail), and the epoch anchor (unique per epoch). The probability of a collision across these three independently unique values is negligible and bounded by the preimage resistance of Keccak256. Any threadId collision detected by the CST Engine is treated as a critical invariant violation under INV-U-006 and triggers an immediate adapter suspension event pending investigation.

2.5 FinalityVector Specification

The **FinalityVector** is a structured descriptor that communicates the finality state of the underlying rail transaction to URIB settlement components. It must be kept current throughout the lifecycle of the CST; stale finality vectors are a common source of incorrect settlement decisions.

```
interface FinalityVector {    railNamespace:        string;    // Must match
the railNamespace of the parent CST.    confirmations:        uint32;    //
Number of block/ledger confirmations observed at time of last query.    // May
only increase within an epoch (INV-U-005).    requiredConfirmations:
uint32;    // Rail-specific finality threshold. For deterministic rails: 1.
// For probabilistic rails: configured per adapter (see per-rail specs,
Section 7).    finalityType:        "probabilistic" | "deterministic" |
"instant";    // "probabilistic": requires requiredConfirmations; reorg risk
remains.    // "deterministic": ledger-closed transactions are final; no reorg
possible.    // "instant": settlement system guarantees finality at time of
acceptance    // (e.g., SEPA SCT Inst, FedNow, CBDC platforms with
instant finality).    estimatedFinalityMs:        uint64;    // Estimated
milliseconds to achieve isFinal=true from time of CST creation.    // Used for
epoch timeout calculations and SLA monitoring.    isFinal:
boolean;    // True if and only if confirmations >= requiredConfirmations AND
// no reorg events have been detected for this threadId.    // This field
governs promotion to SETTLED state. }
```

Rail	Finality Type	Required Confirmations	Typical Finality Time	Reorg Risk
Bitcoin mainnet	Probabilistic	6	~60 minutes	Low (decreases exponentially)
XRP Ledger	Deterministic	1	3–5 seconds	None (consensus-final)
ISO 20022 SEPA SCT Inst	Instant	1 (acceptance)	<10 seconds	None
ISO 20022 SWIFT	Deterministic	1 (settlement msg)	Same day / T+1	None
ISO 20022 FedNow	Instant	1 (acceptance)	<20 seconds	None
Ethereum mainnet	Probabilistic	12	~2.5 minutes	Very low after 12 blocks
Polygon / Arbitrum / Base (L2)	Probabilistic (L2 soft)	L1 state root finality	~15 minutes (L1 finalization)	Low (L2); None after L1 finalization
Stellar mainnet	Deterministic	1	~5 seconds	None
CBDC (retail)	Instant	1	<5 seconds	None
CBDC (wholesale/mBridge)	Deterministic	1	<30 seconds	None

Section 3 — Rail Invariants

3.1 Universal Rail Invariants

The following eight invariants apply to all CSTs, regardless of rail of origin. They are evaluated during Phase 2 (Semantic Validation) of the Invariant Validation Pipeline (Section 3.3). Violation of any universal invariant is a hard rejection condition.

INV-U-001 (Conservation)

For any CST, the following accounting identity must hold: $\text{sum}(\text{sourceDebits}) == \text{sum}(\text{destinationCredits}) + \text{sum}(\text{fees})$. No value may be created or destroyed within a thread. For single-input, single-output TRANSFER threads: $\text{canonicalAmount} + \text{canonicalFee} == \text{sourceDebit}$. For multi-input threads (e.g., UTXO consolidations), all input C_AMTs must sum to the total output C_AMT plus all fees. Zero-amount threads are rejected independently of this invariant.

INV-U-002 (Identity Binding)

Every CST must carry verifiably bound ICTs for both `sourceIdentity` and `destinationIdentity`. An ICT is considered verifiably bound if the Identity Mapping Registry contains a binding proof (a valid signature from the rail-native key corresponding to the ICT) for the originating rail. Unresolvable identities — those for which neither a resolved ICT nor an authorised provisional ICT can be issued — must cause the thread to fail validation before commitment. This invariant cannot be waived by rail adapter configuration.

INV-U-003 (Epoch Anchoring)

Every CST's `commitmentDigest` must be verifiably derivable from the `epochAnchor` of a currently OPEN epoch. Specifically: the epoch anchor committed in the CST's `epochAnchor` field must match the epoch anchor hash of an epoch currently in the OPEN or ACCUMULATING state. CSTs referencing a CLOSING, SETTLED, or ARCHIVED epoch anchor are rejected. Cross-epoch threads are structurally prohibited.

INV-U-004 (Commitment Irreversibility)

Once a `CrossRailCommitment` has been successfully published to the CST Engine, it may not be retracted, modified, or voided by the originating rail adapter acting alone. Any party wishing to reverse a committed CRC must invoke the Bridge Orchestrator's dispute resolution protocol (Section 5.6), which may result in re-settlement, void, or penalty outcomes. Adapters that attempt to delete or overwrite committed CRCs are suspended immediately.

INV-U-005 (Finality Monotonicity)

The `confirmations` field of a `FinalityVector` may only increase within a given epoch. If the rail adapter's finality watcher observes a decrease in confirmation count — indicating a chain reorganisation — it must immediately emit a **Rail Reorg Event** to the CST Engine. The CST Engine then suspends all `threadIds` associated with the affected `RailAnchor`, marks their `FinalityVectors` as `isFinal: false`, and notifies the Bridge Orchestrator for dispute resolution assessment. Suspended threads cannot proceed to `SETTLED` state until their confirmation count is restored above the required threshold.

INV-U-006 (Non-Duplication)

`threadId` values are globally unique within the URIB system. Duplicate `threadIds` are rejected by the CST Engine regardless of rail of origin, epoch, or time of submission. The CST Engine maintains a persistent `threadId` registry (survives across epochs) to enforce this invariant. An adapter that produces a duplicate `threadId` is flagged for investigation; the second submission is discarded with error `E-MAP-010`.

INV-U-007 (Canonical Amount Precision)

All `C_AMT` arithmetic within URIB must use 256-bit unsigned integer mathematics. Rounding must always be toward zero (floor). Any rail adapter that produces a `C_AMT` value larger than the mathematically correct floor result — regardless of cause, including intermediate floating-point conversion, library rounding mode defaults, or incorrect decimal factor application — violates this invariant and creates synthetic value, thereby violating INV-U-001. Adapters must include test vectors demonstrating correct rounding for boundary cases.

INV-U-008 (Semantic Payload Integrity)

The `semanticPayload` bytes field of every CST must hash (via Keccak256) to a value that was committed as part of the `commitmentDigest` computation. Specifically: `commitmentDigest == keccak256(abi.encode(..., keccak256(semanticPayload)))` as specified in the CST schema. Any CST for which the semantic payload cannot be verified against the commitment digest is rejected. This invariant prevents post-hoc modification of rail-specific metadata.

3.2 Rail-Family Invariants

UTXO-Family Rails (Bitcoin, Bitcoin Testnet)

- **INV-UTXO-001 (UTXO Completeness):** All inputs to a UTXO transaction must be accounted for in the `C_AMT` computation. No input UTXO may be referenced in more than one in-flight CST within the same epoch (double-spend prevention at the adapter layer).
- **INV-UTXO-002 (No Legacy Inputs):** Non-SegWit (legacy) transaction inputs are rejected due to transaction ID malleability. All inputs must be SegWit (P2WPKH, P2WSH) or Taproot (P2TR).
- **INV-UTXO-003 (RBF Exclusion):** Transactions with Replace-by-Fee (BIP 125 signalling) set must not be accepted as final and must not transition a CST to `isFinal: true` until the required confirmation threshold is met without replacement.

Account-Family Rails (EVM, XRP, Stellar)

- **INV-ACCT-001 (Nonce Monotonicity):** For account-based rails with nonce/sequence semantics, the adapter must verify that each transaction's nonce/sequence is strictly greater than the previous confirmed nonce/sequence for that account. Out-of-order transactions are rejected.
- **INV-ACCT-002 (Account Existence Proof):** The source account must demonstrably exist on the rail at the time of transaction. Accounts that do not exist cannot be valid sources of value. Adapters must query account existence before producing a CST.
- **INV-ACCT-003 (Balance Sufficiency):** At the time of CST creation, the source account balance must be verified to be sufficient to cover `canonicalAmount + canonicalFee`. Insufficient balance is a structural rejection (not a semantic invariant violation).

Message-Family Rails (ISO 20022)

- **INV-MSG-001 (Message Integrity):** The SHA256 hash of the canonicalised XML message (W3C XML c14n) must be verified and committed in the `semanticPayload`. Any message that fails XML schema validation (XSD) for its declared message type is rejected.
- **INV-MSG-002 (BIC/IBAN Validity):** All BIC codes must conform to ISO 9362 structure. IBAN codes must pass the ISO 13616 check digit algorithm. Messages with structurally invalid BICs or IBANs are rejected with `E-MAP-008`.
- **INV-MSG-003 (R-Transaction Linkage):** Return messages (R-transactions) must carry a `parentThreadId` referencing the original payment's `threadId`. Unlinked R-transactions are rejected.

CBDC Rails

- **INV-CBDC-001 (Regulatory Assertion Anchoring):** Every CBDC CST must include a KYC/AML attestation hash in `semanticPayload`. Attestations with an expiry timestamp earlier than the epoch-open timestamp are rejected.
- **INV-CBDC-002 (Compliance Tier Enforcement):** Transaction amounts must not exceed the per-transaction and per-epoch limits defined for the originating identity's compliance tier. Tier 0 (anonymous) is subject to the most restrictive limits. Tier 2 (verified institutional) has the highest limits, configurable by governance.
- **INV-CBDC-003 (Mint Authorisation):** MINT obligations on CBDC rails require a valid governance authorisation hash signed by the issuing central bank's key. Unauthorised MINT obligations are rejected with severity `HALT`.

3.3 Invariant Validation Pipeline

Every CST submitted to the CST Engine is processed through a three-phase validation pipeline. All phases must pass before the CST is admitted to the active epoch. Phase failures return structured error responses that include the specific invariant or validation rule that was violated.

```
PHASE 1 – STRUCTURAL VALIDATION   Input: raw CST object from rail adapter
Checks:   □ All mandatory fields present and non-null   □ Field types
conform to schema (bytes32 lengths, uint256 ranges, enum values)   □
railNamespace is a registered namespace   □ obligation is a valid
ObligationType value   □ canonicalAmount > 0   □ epochAnchor references
an OPEN or ACCUMULATING epoch   □ If parentThreadId is set: referenced
threadId exists in registry   Output: structurally valid CST or E-MAP-008
(schema violation)   ↓ PASS PHASE 2 – SEMANTIC VALIDATION (Universal
Invariants)   Input: structurally valid CST   Checks (in order):   □ INV-U-
001: Conservation – sum(debits) == sum(credits) + sum(fees)   □ INV-U-002:
Identity Binding – both ICTs verifiably bound   □ INV-U-003: Epoch
Anchoring – epochAnchor matches OPEN epoch   □ INV-U-004: Commitment not
retracted (CRC lifecycle check)   □ INV-U-005: FinalityVector confirmations
non-decreasing   □ INV-U-006: threadId not in duplicate registry   □ INV-
U-007: C_AMT precision – no synthetic value detected   □ INV-U-008:
Semantic payload hashes to committed value in commitmentDigest   Output:
semantically valid CST or E-INV-xxx (invariant violation)   ↓ PASS PHASE 3
– RAIL-SPECIFIC VALIDATION   Input: semantically valid CST + rail family
context   Checks:   □ Apply UTXO-family invariants (INV-UTXO-xxx) for btc:*
namespaces   □ Apply Account-family invariants (INV-ACCT-xxx) for evm:*,
xrp:*, stellar:*   □ Apply Message-family invariants (INV-MSG-xxx) for
iso20022:* namespaces   □ Apply CBDC invariants (INV-CBDC-xxx) for cbdc:*
namespaces   Output: fully valid CST → admitted to epoch OR E-INV-xxx with
family prefix   ↓ PASS CST ADMITTED → assigned to epoch Obligation Graph
```

3.4 Invariant Violation Handling

Invariant	Violation Action	Recovery Path	Severity
INV-U-001 (Conservation)	Reject CST; log violation event	Adapter corrects amount computation; resubmits	REJECT
INV-U-002 (Identity Binding)	Reject CST; flag identity for manual review	Identity registration in ICL; resubmit	REJECT
INV-U-003 (Epoch Anchoring)	Reject CST; return current epoch anchor	Adapter resubmits with correct epoch anchor	REJECT
INV-U-004 (Commitment Irreversibility)	Block reversal; notify Orchestrator	Dispute resolution protocol	REJECT + ALERT
INV-U-005 (Finality Monotonicity)	Suspend affected threadIds; emit Reorg Event	Monitor for reorg resolution; re-evaluate finality	WARN → HALT (if unresolved)

Invariant	Violation Action	Recovery Path	Severity
INV-U-006 (Non-Duplication)	Discard duplicate; log adapter alert	Investigate adapter ID generation logic	REJECT + ALERT
INV-U-007 (C_AMT Precision)	Reject CST; suspend adapter	Adapter code audit; re-certification required	REJECT + SUSPEND
INV-U-008 (Payload Integrity)	Reject CST; flag for forensic review	Adapter recomputes payload and digest; resubmits	REJECT
INV-UTXO-001 (UTXO Completeness)	Reject CST; flag potential double-spend	Wait for conflicting thread to resolve; resubmit	REJECT + ALERT
INV-CBDC-001 (Attestation Anchoring)	Reject CST; require fresh KYC attestation	Refresh attestation hash; resubmit	REJECT
INV-CBDC-003 (Mint Authorisation)	Reject CST; notify governance; halt epoch	Governance review; authorisation issuance	HALT

Section 4 — Canonical Semantic Thread Mapping Rules

4.1 Mapping Philosophy: Rail-Native to CST

Mapping is defined as a deterministic, pure function: `map(nativeTx, railCtx) → CST`. The function must have no side effects — it must not write to any external store, emit any events, or make any network calls. It must be idempotent: calling the mapping function twice with identical inputs must produce identical CSTs. This property is essential for adapter testing (test vectors are authoritative) and for dispute resolution (re-running mapping on disputed transactions must reproduce identical results).

The `railCtx` parameter carries the epoch context (epoch anchor, FX rates, finality thresholds) that is required for amount normalisation and finality assessment. This context is provided by the Bridge Orchestrator and must be treated as immutable within the mapping function's execution scope.

4.2 Mapping Rule Set

CST-M-001 (Amount Normalisation)

The canonical amount is computed as:

```
C_AMT = floor(nativeAmount × 10^(18 - nativeDecimals) × fxRateNumerator /
fxRateDenominator)  Overflow protection:  - Intermediate products MUST be
computed in 512-bit space before truncation to 256-bit.  - If the
intermediate product exceeds 2^256 - 1, throw E-MAP-002 (amount overflow).
Zero-amount rejection:  - If nativeAmount == 0, throw E-MAP-002 with sub-
code ZERO_AMOUNT.  - Zero-amount threads are structurally invalid and cannot
represent any settlement obligation.
```

CST-M-002 (Identity Normalisation)

Every rail-native address or account identifier must be mapped to an ICT via the Identity Mapping Registry (ICL Layer 4). The mapping procedure is:

1. Normalise the raw address to its canonical form for the rail (e.g., lowercase hex for EVM, base58check for Bitcoin, G-address for Stellar).
2. Query the ICL registry for an existing ICT binding for `(railNamespace, canonicalAddress)`.
3. If found: return the bound ICT.

4. If not found: attempt to derive a provisional ICT using the address's public key if extractable.

Provisional ICTs are flagged with `provisional: true` and are subject to binding confirmation within 2 epochs.

5. If a provisional ICT cannot be derived (e.g., contract address without ABI commitment): throw

`E-MAP-003`.

ICT format: `<rail-namespace>:<encoded-canonical-pubkey-or-hash>`. The encoded component is the lower-case hex of the 20-byte Keccak256 digest of the canonical public key bytes.

CST-M-003 (Obligation Classification)

The mapping function must classify every transaction into exactly one `ObligationType` using the following decision tree:

```
Is the transaction a provable value destruction (burn) with no output?  YES
→ BURN (requires CBDC/governance authorisation for non-zero amounts) Is the
transaction a provable value creation (mint) with no input?  YES → MINT
(requires CBDC/governance authorisation; see INV-CBDC-003) Does the
transaction lock value conditionally or by time?  YES → ESCROW      Examples:
Bitcoin HTLC, XRP native Escrow, CBDC conditional transfer,          ISO
20022 pain.001 pending acceptance Does the transaction span ≥2 rails
atomically?  YES → ATOMIC (two-phase commit required; see Section 5.4) Does
the transaction simultaneously exchange value on the same rail?  YES → SWAP
(DEX trades on XRP, Stellar, EVM AMMs) Default (standard value transfer):
→ TRANSFER
```

CST-M-004 (Thread Chaining)

When a native transaction is logically part of a payment sequence — such as a Lightning Network route with multiple hops, an XRP Payment Path traversing multiple offer books, or an ISO 20022 cover payment with an underlying customer payment — individual CSTs must be chained via the `parentThreadId` field.

Chain validation rules:

- The `parentThreadId` must reference a `threadId` that exists in the CST Engine registry.
- The parent thread must be in the same epoch (same `epochAnchor`).
- Cycle detection: the adapter must verify that following the `parentThreadId` chain does not produce a cycle. Cycles are rejected with `E-MAP-005`. Maximum chain depth is 32 threads (configurable).
- The aggregate `C_AMT` of a chain must satisfy the conservation invariant (INV-U-001) at each link in the chain.

CST-M-005 (Semantic Payload Encoding)

The `semanticPayload` is ABI-encoded per rail family. Encoding must be deterministic and produce identical bytes for identical inputs.

Rail Family	Payload Contents	Encoding
UTXO (Bitcoin)	scriptPubKey type (enum), input TXID set (bytes32[]), output index set (uint32[]), locktime (uint32), version (uint32)	ABI encode(tuple)
Account (EVM)	nonce (uint64), to (bytes20), calldata hash (bytes32), contract ABI commitment (bytes32 null), chainId (uint32)	ABI encode(tuple)
Account (XRP)	account sequence (uint32), destination tag (uint32 null), offer sequence (uint32 null), payment path hash (bytes32 null)	ABI encode(tuple)
Account (Stellar)	source sequence (uint64), muxed sub-ID (uint64 null), operation type (enum), asset code (bytes12), asset issuer (bytes32)	ABI encode(tuple)
Message (ISO 20022)	XML c14n hash (bytes32), message type (bytes8, e.g., "pacs.008"), BIC pair (bytes22), clearing reference (bytes35)	ABI encode(tuple)
CBDC	KYC/AML attestation hash (bytes32), attestation expiry (uint64), compliance tier (uint8), regulatory domain (bytes8), programmability condition hash (bytes32 null)	ABI encode(tuple)

CST-M-006 (Finality Integration)

The `FinalityVector` embedded in a CST must be refreshed before the thread is promoted to SETTLED state. Adapters must implement one of two finality watching strategies:

- **Polling:** Query the rail node at a configurable interval (default: 15 seconds for probabilistic rails, 5 seconds for deterministic rails). Update the `FinalityVector` in-place. Emit a `FINALITY_UPDATED` event to the CST Engine when `isFinal` transitions to `true`.
- **Event-Driven:** Subscribe to rail node block/ledger events. Update `FinalityVector` and emit `FINALITY_UPDATED` on each relevant event. Preferred for deterministic rails and high-throughput adapters.

A CST whose `FinalityVector` has not been refreshed within `finality_staleness_threshold` milliseconds (default: 120,000 ms) before EPOCH CLOSING is marked STALE and carries over to the next epoch.

4.3 Mapping Error Codes

Error Code	Description	Recovery Action
E-MAP-001	Unknown or unregistered rail namespace	Register namespace in RailNamespace Registry; retry
E-MAP-002	Amount overflow or zero-amount rejection	Check nativeAmount and decimal factors; fix overflow in 512-bit intermediate
E-MAP-003	Identity not found; provisional ICT cannot be derived	Register identity in ICL; ensure canonical public key is available
E-MAP-004	Invalid obligation type; transaction does not match any ObligationType	Review CST-M-003 decision tree; check transaction structure
E-MAP-005	Thread chain cycle detected	Review payment path construction; remove circular references
E-MAP-006	Semantic payload encoding failure	Verify ABI encoding library; check field types match schema
E-MAP-007	Finality timeout; FinalityVector not updated within staleness threshold	Check rail node connectivity; restart finality watcher; consider carry-forward
E-MAP-008	Epoch mismatch; epochAnchor does not match any OPEN epoch	Re-fetch current epoch anchor from Bridge Orchestrator; resubmit
E-MAP-009	Schema violation; mandatory field missing or wrong type	Review CST schema definition; fix adapter field construction
E-MAP-010	Duplicate threadId; already exists in registry	Investigate threadId computation; check nativeTxId uniqueness on rail
E-MAP-011	FX rate stale; oracle commitment exceeds staleness threshold	Wait for next epoch (new oracle commitment); do not normalise with stale rate
E-MAP-012	Commitment digest mismatch; payload does not match committed hash	Recompute commitmentDigest; ensure payload not modified after digest computation

Section 5 — Settlement Equivalence Logic

5.1 Settlement Equivalence Proof (SEP) Overview

A **Settlement Equivalence Proof (SEP)** is a cryptographic attestation produced by the Settlement Equivalence Engine that a set of CSTs across two or more rails (or one rail for single-rail settlement) collectively satisfy a settlement obligation represented by a node in the Obligation Graph. The SEP is the core output of the cross-rail settlement process and the primary artefact consumed by downstream systems (treasury management, regulatory reporting, custodial accounting).

A SEP is valid if and only if: (1) all referenced CSTs are admitted to the same epoch; (2) the aggregate canonical value satisfies the obligation amount; (3) all FinalityVectors show `isFinal: true` or fall within the epoch's finality grace period; and (4) the aggregate signature from the validator set is valid against the SEP data.

5.2 SEP Data Structure

```
interface SettlementEquivalenceProof {    sepId:                bytes32;    // Unique SEP identifier.    // Computed as: keccak256(abi.encode(epochAnchor, threads, obligationHash))    epochAnchor:            bytes32;    // The epoch root hash under which all contributing CSTs were committed.    threads:                bytes32[];    // Ordered array of threadIds of all CSTs contributing to this SEP.    // Must contain at least 1 threadId (single-rail settlement is valid).    // For ATOMIC obligations: must contain ≥2 threadIds from ≥2 distinct railNamespaces.    totalCanonicalValue:    C_AMT;    // Sum of canonicalAmount across all threads. Must equal obligationNode.requiredValue.    // Computed as: sum(cst.canonicalAmount for cst in threads)    obligationHash:        bytes32;    // keccak256(abi.encode(obligationGraphNode)) - binds SEP to a specific    // Obligation Graph node. Prevents SEP replay across different obligations.    railAnchors:            RailAnchor[];    // On-chain anchoring references, one per participating rail.    // RailAnchor { railNamespace: string; txId: bytes32; blockHeight: uint64 }    aggregateSignature:    bytes;    // Threshold m-of-n signature from the bridge validator set, OR    // a ZK proof of correct SEP computation (if ZK mode is enabled per deployment).    // For Schnorr threshold: 64-byte aggregate signature.    // For BLS: 48-byte BLS12-381 aggregate signature.    finalityVectors:        FinalityVector[];    // One FinalityVector per participating CST, in the same order as threads[].    // All must have isFinal: true, OR the SEP must be within the finality grace period.    settlementTimestamp:    uint64;    // Unix epoch milliseconds at time of SEP production by the SEE.    status:                "PENDING" | "COMPLETE" | "DISPUTED" | "VOIDED";    // PENDING: SEP created, awaiting validator threshold signature.    // COMPLETE: All validators signed; SEP is authoritative.    // DISPUTED: A dispute has been raised against this SEP (Section 5.6).    // VOIDED: SEP was invalidated (reorg, dispute resolution outcome). }
```

5.3 SEP Generation Algorithm

The Settlement Equivalence Engine executes the following algorithm to produce a SEP for a given Obligation Graph node. This algorithm runs at epoch CLOSING for all PENDING obligation nodes.

```
FUNCTION generateSEP(obligationNode: ObligationGraphNode, epoch: BridgeEpoch)
→ SEP:    STEP 1: Collect contributing CSTs    threads ←
CST_Engine.getThreadsByObligationNode(obligationNode.id)    ASSERT
len(threads) ≥ 1, throw E-SEP-001 if empty    STEP 2: Verify epoch
consistency (INV-U-003)    FOR EACH cst IN threads:    ASSERT
cst.epochAnchor == epoch.anchorHash, throw E-SEP-002    STEP 3: Conservation
verification (INV-U-001)    totalValue ← sum(cst.canonicalAmount FOR cst IN
threads)    totalFees ← sum(cst.canonicalFee FOR cst IN threads)
ASSERT totalValue == obligationNode.requiredValue, throw E-SEP-003    STEP 4:
Finality verification (CST-M-006)    FOR EACH cst IN threads:    fv ←
adapter[cst.railNamespace].queryFinality(cst.finalityVector.railAnchor)
IF NOT fv.isFinal:    IF epoch.timeRemaining < fv.estimatedFinalityMs:
MARK thread for carry-forward; REMOVE from this SEP    ELSE:
WAIT up to finality_grace_period_ms    ASSERT all remaining threads have
isFinal: true    STEP 5: Construct obligationHash    obligationHash ←
keccak256(abi.encode(obligationNode))    STEP 6: Collect RailAnchors
railAnchors ← []    FOR EACH distinct railNamespace IN
threads[].railNamespace:    anchor ←
CST_Engine.getRailAnchor(railNamespace, epoch.id)
railAnchors.append(anchor)    STEP 7: Produce aggregate signature
sepPayload ← abi.encode(epochAnchor, threads[].threadId, totalCanonicalValue,
obligationHash)    aggregateSignature ←
validatorSet.thresholdSign(keccak256(sepPayload))    // Requires m-of-n
validators to sign. Default: 3-of-5.    STEP 8: Construct and publish SEP
sep ← SettlementEquivalenceProof {    sepId:
keccak256(abi.encode(epoch.anchorHash, threads[].threadId, obligationHash)),
epochAnchor:    epoch.anchorHash,    threads:
threads[].threadId,    totalCanonicalValue: totalValue,
obligationHash:    obligationHash,    railAnchors:    railAnchors,
aggregateSignature: aggregateSignature,    finalityVectors:
threads[].finalityVector,    settlementTimestamp: now_ms(),    status:
"COMPLETE"    }    CST_Engine.publishSEP(sep)    EMIT SETTLED event {
sepId, obligationNode.id, epoch.id }    RETURN sep
```

5.4 Cross-Rail Atomic Settlement

ATOMIC obligation threads require a two-phase commit protocol to ensure that value movements on two or more rails are either all committed or all aborted — no partial settlement is permitted. The Bridge Orchestrator coordinates the protocol; rail adapters implement the locking and commitment mechanics.

Phase 1 — Prepare

6. The Bridge Orchestrator identifies all rail adapters participating in the ATOMIC obligation.
7. It sends a PREPARE instruction to each adapter, specifying the CST to be settled on that rail.
8. Each adapter locks the relevant funds on its rail (e.g., publishes an HTLC on Bitcoin, creates an XRP Escrow, deploys a time-locked contract on EVM, or reserves balance in a CBDC system).
9. Each adapter returns a **prepare commitment hash**: `keccak256(railNamespace || lockedAmount || lockExpiry || preimageHash)`.
10. The Bridge Orchestrator collects prepare commitment hashes from all participating adapters.

Phase 2 — Commit or Abort

11. If all prepare commitment hashes are received within the **prepare timeout** (configurable per obligation; default: 180 seconds):
12. If any adapter fails to return a prepare commitment hash within the prepare timeout, OR if any adapter returns a prepare failure:

Rail Family	Prepare Mechanism	Prepare Timeout	Abort Mechanism
Bitcoin (UTXO)	HTLC via OP_IF + OP_CHECKLOCKTIMEVERIFY	180s	HTLC timeout path
XRP Ledger	Native XRP Escrow (EscrowCreate)	180s	EscrowCancel after expiry
ISO 20022	pain.001 pending acceptance state	600s	cancellation (camt.056)
CBDC	Conditional transfer / programmable lock	300s	Platform-specific release call
EVM	Time-locked escrow contract (ERC-compatible)	180s	contract.abort() after timeout

Rail Family	Prepare Mechanism	Prepare Timeout	Abort Mechanism
Stellar	Claimable Balance with time-bound claimant	180s	Reclaimable after expiry

5.5 Settlement Equivalence Rules Table

Rule ID	Scenario	Required SEP Fields	Validity Window	Notes
SEP-R-001	Single-rail simple transfer	1 thread, 1 railAnchor, isFinal=true	Current epoch	No atomic protocol needed
SEP-R-002	Two-rail atomic swap	2 threads from 2 namespaces, 2 railAnchors, both isFinal=true, ATOMIC obligation	Current epoch (prepare timeout applies)	Two-phase commit required
SEP-R-003	Multi-hop payment path	N threads with parentThreadId chain, N railAnchors, all isFinal=true	Current epoch	CST-M-004 chain validation required
SEP-R-004	CBDC mint-and-transfer	MINT thread + TRANSFER thread, governance authorisation hash, CBDC compliance attestation	Current epoch	INV-CBDC-003 governance auth required
SEP-R-005	ISO 20022 cover payment	pacs.009 TRANSFER thread + pacs.008 TRANSFER thread with parentThreadId link	Current epoch	BIC pair must match across threads
SEP-R-006	Lightning-to-EVM bridge	HTLC ESCROW thread (BTC Lightning) + TRANSFER thread (EVM), ATOMIC obligation	Current epoch; HTLC expiry must exceed epoch close time	Preimage reveal required for COMMIT phase
SEP-R-007	Stellar path payment	N threads (one per path hop) with chain linkage, 1 Stellar railAnchor	Current epoch	Stellar deterministic finality: isFinal after 1 ledger close

5.6 Dispute Resolution Protocol

Dispute Type 1 — Amount Dispute

Definition: The canonical amount reported by two or more adapters for the same cross-rail leg disagrees. Detected when SEP conservation check (INV-U-001) fails across rails.

Detection: Automated — SEE detects C_AMT mismatch during Step 3 of SEP generation algorithm.

Evidence Requirements: Both adapters must submit their raw mapping inputs (`nativeTx`, `railCtx`) and their computed CST. The Orchestrator re-runs the canonical mapping function independently.

Adjudication: The Orchestrator's mapping result is authoritative. The adapter whose output deviates is flagged as faulty. If both deviate, human governance is invoked.

Resolution Outcomes: Correct adapter's value is used for SEP; faulty adapter is suspended and slashed (Section 10.4). Re-settlement with corrected C_AMT is issued.

Dispute Type 2 — Finality Dispute

Definition: A chain reorganisation on a probabilistic rail invalidates a CST that has already been marked `isFinal: true` and included in a completed SEP.

Detection: Finality watcher emits a Rail Reorg Event (INV-U-005 violation). The CST Engine cross-references affected threadIds against completed SEPs.

Evidence Requirements: The adapter must provide the competing chain header and proof of greater accumulated work (for PoW rails) or validator signatures (for PoS rails).

Adjudication: Bridge Orchestrator suspends the affected SEP (status → DISPUTED). Validator set votes on reorg validity. If reorg confirmed: SEP is VOIDED; re-settlement is initiated in the next epoch.

Resolution Outcomes: SEP voided, re-settled, or partially re-settled depending on which rails were affected. Bond of affected adapter is slashed proportionally.

Dispute Type 3 — Identity Dispute

Definition: An ICT binding is contested — a party claims the ICT attributed to their native address belongs to a different entity, or that a binding proof is forged.

Detection: Manual report by affected party, or automated detection of cryptographic inconsistency in binding proofs.

Evidence Requirements: Challenger must provide a competing binding proof (valid signature from the claimed key) that postdates the disputed binding.

Adjudication: Three-tier escalation: (1) Automated — Orchestrator verifies both binding proofs cryptographically; (2) Validator vote — if cryptographic outcome is ambiguous; (3) Human governance committee — for complex cases involving legal identity.

Resolution Outcomes: ICT rebinding with version increment; in-flight threads using old binding are suspended and re-evaluated; SEPs referencing disputed ICT are held in DISPUTED state pending resolution.

Section 6 — Identity Continuity Layer (ICL)

6.1 Identity Continuity Token (ICT) Architecture

The **Identity Continuity Token (ICT)** is a 32-byte value that provides a persistent, rail-agnostic identifier for a participant within the URIB system. It is the primary identity primitive: all CSTs, SEPs, and Obligation Graph nodes reference participants via their ICTs, never via rail-native addresses.

The ICT is computed as:

```
ICT = keccak256(abi.encodePacked(    railNamespace,          // string: e.g.
"btc:mainnet"    canonicalPubkeyHash,    // bytes32: keccak256 of canonical
public key bytes    identityVersion      // uint32: monotonically increasing
version counter ))
```

The `identityVersion` starts at 0 and increments by 1 on every key rotation. Old ICT versions remain valid for in-flight threads during the migration window; only the current version is eligible for new CSTs. This enables key rotation without disrupting active settlements.

For accounts without extractable public keys (e.g., smart contracts, composite accounts), the `canonicalPubkeyHash` is computed from the account's commitment data: ABI hash for EVM contracts, LEI + BIC hash for ISO 20022 institutional accounts, or regulatory identity assertion hash for CBDC accounts.

6.2 Identity Mapping Registry

The Identity Mapping Registry is a deterministic key-value store maintained by the ICL. It maps ICTs to full identity records. The registry is append-only for bindings and version-only-increasing for updates. Deletions are not permitted; deprecated bindings are marked EXPIRED.

Each registry entry contains the following fields:

Field	Type	Description
<code>ict</code>	<code>bytes32</code>	Primary key. ICT value as computed above.
<code>namespaceBindings</code>	<code>Map<RailNamespace, NativeAddress></code>	All rail-native addresses associated with this ICT, keyed by namespace.
<code>canonicalPubkeyCommitment</code>	<code>bytes32</code>	Hash of the canonical public key used in ICT derivation.
<code>identityVersion</code>	<code>uint32</code>	Current version. Incremented on key rotation.
<code>bindingProofs</code>	<code>Map<RailNamespace, bytes></code>	Rail-native signatures proving ownership of each bound address.
<code>kycAmlAttestationHash</code>	<code>bytes32 null</code>	Hash of KYC/AML attestation document. Required for CBDC rails.
<code>complianceTier</code>	<code>uint8 null</code>	CBDC compliance tier: 0=anonymous, 1=verified individual, 2=verified institutional.
<code>createdEpochAnchor</code>	<code>bytes32</code>	Epoch anchor at time of first registration.
<code>updatedEpochAnchor</code>	<code>bytes32</code>	Epoch anchor at time of most recent update.
<code>status</code>	<code>enum</code>	ACTIVE PROVISIONAL ROTATING EXPIRED SUSPENDED

6.3 Identity Mapping Templates

Template ICT-BTC — Bitcoin Identity Mapping

Bitcoin supports four address types, all of which must be supported. The canonical public key for ICT derivation is always the underlying secp256k1 compressed public key (33 bytes), regardless of address type.

```
// P2PKH (Legacy): 1-prefixed base58check address canonicalPubkeyHash =  
keccak256(compressed_secp256k1_pubkey) ICT =  
keccak256(abi.encodePacked("btc:mainnet", canonicalPubkeyHash,  
identityVersion)) // P2SH (Script Hash): 3-prefixed. Map to the redeemScript  
hash. canonicalPubkeyHash = keccak256(redeemScript) ICT =  
keccak256(abi.encodePacked("btc:mainnet", canonicalPubkeyHash,  
identityVersion)) // P2WPKH (Native SegWit): bclq-prefixed. Same as P2PKH  
but key extracted from witness. canonicalPubkeyHash =  
keccak256(compressed_secp256k1_pubkey) ICT =  
keccak256(abi.encodePacked("btc:mainnet", canonicalPubkeyHash,  
identityVersion)) // P2TR (Taproot): bclp-prefixed. Distinguish key-path vs.  
script-path. // Key-path: tweaked internal key. Use internal key (x-only,  
32 bytes) as canonical. // Script-path: MAST root hash used as  
canonicalPubkeyHash. // NOTE: key-path and script-path Taproot outputs from  
the same key get separate ICTs // unless explicitly linked in the  
registry. canonicalPubkeyHash_keypath = keccak256(internal_only_pubkey) //  
32 bytes ICT_keypath = keccak256(abi.encodePacked("btc:mainnet",  
canonicalPubkeyHash_keypath, identityVersion))
```

Template ICT-XRP — XRP Ledger Identity Mapping

```
// Classic XRP Address (base58check AccountID — RIPEMD160(SHA256(pubkey)))  
accountId = RIPEMD160(SHA256(compressed_secp256k1_pubkey | ed25519_pubkey))  
canonicalPubkeyHash = keccak256(accountId) // convert to 32-byte hash ICT =  
keccak256(abi.encodePacked("xrp:mainnet", canonicalPubkeyHash,  
identityVersion)) // X-Address format (includes destination tag): X-prefixed  
// X-Address encodes (classicAddress, destinationTag, isTestnet) // Sub-  
identities with different destination tags map to child ICTs: parentICT = ICT  
derived from classicAddress as above childICT =  
keccak256(abi.encodePacked("xrp:mainnet", keccak256(accountId ||  
destinationTag), identityVersion)) // childICT registry entry must carry  
parentICT in a parentBinding field.
```

Template ICT-ISO20022 — ISO 20022 Identity Mapping

```
// Institutional account: BIC (8 or 11 chars) + IBAN composite bicBytes =  
UTF8_encode(BIC) // 8 or 11 bytes ibanBytes = UTF8_encode(IBAN)  
// variable canonicalPubkeyHash = keccak256(bicBytes || ibanBytes) ICT =  
keccak256(abi.encodePacked("iso20022:sepa", canonicalPubkeyHash,  
identityVersion)) // LEI binding (Legal Entity Identifier, 20 chars ISO  
17442): leiBytes = UTF8_encode(LEI) // If LEI is present, augment the  
canonical hash: canonicalPubkeyHash = keccak256(bicBytes || ibanBytes ||  
leiBytes) ICT = keccak256(abi.encodePacked("iso20022:sepa",  
canonicalPubkeyHash, identityVersion)) // Provisional ICT (LEI absent): //  
Issue provisional ICT with status=PROVISIONAL. Valid for 2 epochs. // Binding  
proof: institution must sign a challenge with their HSM key. // Provisional  
ICTs carry kycAmlAttestationHash = null; CBDC rails reject provisional ICTs.
```

Template ICT-CBDC — CBDC Identity Mapping

```
// Regulatory Identity Assertion (RIA) approach // RIA is a signed assertion
from the national KYC registry: // RIA = sign_by_registry(kycSubjectId ||
complianceTier || expiryTimestamp)  riaHash =
keccak256(abi.encode(kycSubjectId, complianceTier, expiryTimestamp,
registry_sig)) canonicalPubkeyHash = keccak256(cbdac_account_pubkey ||
riaHash) ICT = keccak256(abi.encodePacked("cbdc:eur:ecb",
canonicalPubkeyHash, identityVersion)) // Compliance Tiers: // Tier 0:
anonymous low-value. No KYC. Per-transaction limit: 50 CBDC units. //
Per-epoch limit: 500 CBDC units. RIA hash = null. // Tier 1: verified
individual. Full personal KYC. Limits per jurisdiction. // RIA hash
required; expiry ≤ 365 days. // Tier 2: verified institutional. AML-
certified entity. Highest limits. // RIA hash required; LEI binding
required; expiry ≤ 90 days. // Tier enforcement: adapter must verify
complianceTier from registry entry // before normalising any CBDC
transaction. Expired RIAs fail INV-CBDC-001.
```

Template ICT-EVM — EVM Identity Mapping

```
// Externally Owned Account (EOA): secp256k1 address (last 20 bytes of
keccak256(pubkey)) canonicalPubkeyHash =
keccak256(uncompressed_secp256k1_pubkey) // 65-byte pubkey ICT =
keccak256(abi.encodePacked("evm:1:mainnet", canonicalPubkeyHash,
identityVersion)) // Smart Contract Account: no native public key // ABI
commitment: keccak256 of the deployed contract bytecode + ABI JSON
abiCommitment = keccak256(deployedBytecode || abiJsonBytes)
canonicalPubkeyHash = keccak256(contractAddress || abiCommitment) ICT =
keccak256(abi.encodePacked("evm:1:mainnet", canonicalPubkeyHash,
identityVersion)) // EIP-1271 (Contract Signature Verification): // For
contract-held identities, binding proof requires calling
isValidSignature(hash, sig) // on the contract. The URIB ICL must verify EIP-
1271 compliance before issuing ICT. // Adapters must cache the EIP-1271
result with a validity window of 1 epoch. // If the contract does not
implement EIP-1271, provisional ICT is issued.
```

Template ICT-STEELLAR — Stellar Identity Mapping

```
// G-address: Ed25519 public key in base32 Stellar format
ed25519_pubkey_bytes = decode_base32_stellar(G_address) // 32 bytes
canonicalPubkeyHash = keccak256(ed25519_pubkey_bytes) ICT =
keccak256(abi.encodePacked("stellar:mainnet", canonicalPubkeyHash,
identityVersion)) // M-address (Muxed Account): encodes (G-address, uint64
sub-ID) // Each muxed account sub-ID is a distinct sub-identity: parentICT =
ICT derived from G-address as above subIdHash =
keccak256(abi.encodePacked(ed25519_pubkey_bytes, muxed_sub_id)) childICT =
keccak256(abi.encodePacked("stellar:mainnet", subIdHash, identityVersion)) //
childICT registry entry must carry parentICT as parentBinding. // Binding
proof: parent G-address signature over (childICT || subId).
```

6.4 Cross-Rail Identity Resolution

Given an ICT, the cross-rail resolver returns all known native addresses for that identity across all registered rails. This is the primary query used by the SEE when constructing SEPs and by compliance tooling for transaction monitoring.

```
FUNCTION resolveICT(ict: bytes32) → IdentityResolutionResult:  entry ←
ICL_Registry.lookup(ict)  IF entry == null:  // ICT not found in
registry  RETURN { status: "UNREGISTERED", namespaceBindings: {}, note:
"Provisional ICT possible" }  IF entry.status == "PROVISIONAL":  RETURN
{  status: "PROVISIONAL",  namespaceBindings:
entry.namespaceBindings,  expiresInEpochs: 2 - (currentEpoch -
entry.createdEpoch),  note: "Binding confirmation pending"
}  IF entry.status == "SUSPENDED":  EMIT identity_dispute_alert(ict)
RETURN { status: "SUSPENDED", note: "Identity under dispute – contact URIB
governance" }  RETURN {  status: "ACTIVE",
namespaceBindings: entry.namespaceBindings,  // Map  identityVersion:
entry.identityVersion,  complianceTier: entry.complianceTier,  //
null for non-CBDC identities  bindingProofs: entry.bindingProofs  }
```

6.5 Key Rotation and ICT Migration

When a participant rotates their key on any rail (e.g., Bitcoin address rotation, EVM key compromise recovery, Stellar account re-keying), the ICT version is incremented and a new ICT is registered. The migration procedure is:

13. **Initiation:** Participant submits a key rotation request to the ICL, signed by both the old key and the new key (dual-signature proof of knowledge and intent).
14. **Version Increment:** ICL increments `identityVersion` from V to V+1. New ICT (version V+1) is derived and registered as ACTIVE. Old ICT (version V) is marked ROTATING.
15. **Migration Window:** During the migration window (default: 2 epochs, configurable), both ICT versions are valid. In-flight threads that reference the old ICT version are permitted to complete. New CSTs must use the new ICT version.
16. **Expiry:** After the migration window, the old ICT version is marked EXPIRED. Any CST referencing an EXPIRED ICT version is rejected.
17. **Cross-Rail Consistency:** Key rotation on one rail does not automatically update other rail namespace bindings. Each binding must be individually updated with a new binding proof for the new key. Until updated, other rail bindings continue to reference the new ICT version via the registry update.

Section 7 — Per-Rail Adapter Specifications

7.1 Bitcoin (BTC) Rail Adapter

Bitcoin operates on a UTXO (Unspent Transaction Output) model. There are no accounts; value is held in outputs from prior transactions and consumed as inputs to new transactions. Every BTC transaction references one or more prior UTXOs as inputs and produces one or more new UTXOs as outputs. The difference between input value and output value is the miner fee, which must be captured in `canonicalFee`.

The Bitcoin adapter must support SegWit (P2WPKH, P2WSH) and Taproot (P2TR) transaction formats. Legacy (P2PKH, P2SH) inputs are rejected per INV-UTXO-002 due to transaction ID malleability. The Lightning Network sub-adapter handles off-chain HTLC settlements and maps them to ESCROW CSTs.

Configuration Parameter	Type	Default	Description
<code>btc_node_rpc_url</code>	string	—	Bitcoin Core RPC endpoint
<code>btc_required_confirmations</code>	uint32	6	Confirmations for <code>isFinal=true</code>
<code>btc_op_return_prefix</code>	bytes4	0x55524942 ("URIB")	OP_RETURN magic prefix for URIB commitments
<code>btc_min_relay_fee_sat_vb</code>	uint32	1	Minimum relay fee in sat/vbyte for commitment transactions
<code>btc_rbf_rejection</code>	boolean	true	Reject RBF-signalling transactions as non-final

Hashing & Commitment: Transaction IDs are computed as `SHA256d(serialised_tx)` (double SHA256). SegWit transaction IDs (`txid`) use the non-witness serialisation; witness transaction IDs (`wtxid`) use full witness serialisation. URIB always uses `txid` for CST `threadId` computation. Commitment is published via an OP_RETURN output: `OP_RETURN <URIB_prefix> <commitmentDigest>` (44 bytes total, within OP_RETURN data limit).

Amount Normalisation (worked example):

```
nativeAmount = 50,000,000 satoshis (0.5 BTC) nativeDecimals = 8 conversion
factor = 10^(18-8) = 10^10 C_AMT (pre-FX) = 50,000,000 × 10^10 =
500,000,000,000,000,000 (5×10^17) epochFxRate = 95,000 (USD/BTC equivalent
in SDR, expressed as integer) = fxRateNumerator: 95000,
fxRateDenominator: 1 C_AMT = floor(500,000,000,000,000,000 × 95000 / 1)
= 47,500,000,000,000,000,000 URIB base units ≈ 47,500 × 10^18 URIB
units (representing ~$47,500 in SDR terms)
```

Finality Model: Bitcoin finality is probabilistic. At 0 confirmations, the transaction is in the mempool and can be replaced (if RBF-signalling) or double-spent. At 1 confirmation, the risk is approximately $1/(2^k)$ for k additional confirmations needed. URIB requires 6 confirmations (~60 minutes average) for

`isFinal: true.`

Lightning Network Sub-Adapter: HTLC settlements on Lightning are mapped to ESCROW CSTs. The HTLC preimage hash serves as the escrow condition. When the preimage is revealed (payment successful), the ESCROW CST is promoted to TRANSFER. When the HTLC times out (payment failed), the ESCROW CST is marked ABORTED. Lightning channel capacity proofs are included in `semanticPayload`.

Common Integration Pitfalls:

- **RBF transactions:** BIP 125 signals in the `nSequence` field (value < 0xFFFFFFFF) indicate replaceability. Never mark such transactions as final before 6 confirmations even if they appear confirmed in the mempool.
- **Pre-SegWit inputs:** Legacy P2PKH and P2SH inputs produce malleable txids. Reject these per INV-UTXO-002. Do not accept any transaction that spends a legacy UTXO.
- **UTXO set consistency:** The adapter must maintain a local UTXO snapshot per epoch to detect in-epoch double-spend attempts. Do not rely solely on the Bitcoin node's mempool, which may not contain conflicting transactions seen by other nodes.
- **OP_RETURN size:** Ensure commitment encoding does not exceed 80 bytes (the standard OP_RETURN data limit). The URIB prefix (4 bytes) + commitmentDigest (32 bytes) = 36 bytes — well within limit.

```

// Bitcoin normaliseTransaction pseudocode function
normaliseTransaction(nativeTx: BitcoinRawTransaction, railCtx: RailContext):
CST { // 1. Reject legacy inputs for (input of nativeTx.inputs) { if
(!isSegWit(input) && !isTaproot(input)) { throw new MappingError("E-
MAP-008", "Legacy non-SegWit input rejected: " + input.txid) } } //
2. Compute txid (non-witness serialisation, double SHA256) const txid:
bytes32 = SHA256d(serializeNoWitness(nativeTx)) // 3. Compute threadId
const threadId: bytes32 = keccak256(abi.encode("btc:mainnet", txid,
railCtx.epochAnchor)) // 4. Reject RBF-signalling if configured if
(railCtx.config.btc_rbf_rejection && isRBFSignalling(nativeTx)) {
fv.isFinal = false // Will never be final until 6 confirmations } // 5.
Sum inputs and outputs for conservation check const totalIn: bigint =
sumUTXOValues(nativeTx.inputs, railCtx.utxoSnapshot) const totalOut: bigint
= sumOutputValues(nativeTx.outputs) const fee: bigint = totalIn -
totalOut // Main transfer output (largest non-OP_RETURN output) const
amount: bigint = selectTransferAmount(nativeTx.outputs) // 6. Normalise
amounts const canonicalAmount: C_AMT = normaliseAmount(amount, "satoshi")
const canonicalFee: C_AMT = normaliseAmount(fee, "satoshi") // 7.
Normalise identities const sourceIdentity: ICT =
normaliseIdentity(extractSourceAddress(nativeTx)) const
destinationIdentity: ICT = normaliseIdentity(extractDestAddress(nativeTx))
// 8. Classify obligation const obligation: ObligationType =
classifyObligation(nativeTx) // CST-M-003 // 9. Encode semantic payload
(CST-M-005) const scriptType: ScriptType =
detectScriptType(nativeTx.outputs[0]) const semanticPayload: bytes =
abi.encode(scriptType, txid, nativeTx.locktime, nativeTx.version) // 10.
Compute commitment digest const commitmentDigest: bytes32 =
keccak256(abi.encode(threadId, railCtx.epochAnchor, sourceIdentity,
destinationIdentity, canonicalAmount, canonicalFee, obligation,
keccak256(semanticPayload))) // 11. Build FinalityVector const fv:
FinalityVector = queryFinality({ railNamespace: "btc:mainnet", txId: txid })
return { threadId, epochAnchor: railCtx.epochAnchor, railNamespace:
"btc:mainnet", sourceIdentity, destinationIdentity, canonicalAmount,
canonicalFee, obligation, semanticPayload, timestamp: now_ms(),
finalityVector: fv, commitmentDigest, parentThreadId: null } }

```

7.2 XRP Ledger Rail Adapter

The XRP Ledger is an account-based blockchain with native support for decentralised exchange (DEX) functionality via Offers, multi-currency payments via Payment Paths, and native escrow functionality. The consensus algorithm (Ripple Protocol Consensus Algorithm / XRP Ledger Consensus Protocol) achieves deterministic finality within 3–5 seconds, making it suitable for real-time settlement scenarios. XRP amounts are denominated in drops (1 XRP = 1,000,000 drops). IOU amounts for issued currencies (token balances held via trust lines) are denominated in the issuer's currency and require FX conversion. The adapter must handle both XRP (native) and IOU amounts, with IOU amounts requiring trust line path resolution before canonical value can be determined.

Configuration Parameter	Type	Default	Description
<code>xrp_node_ws_url</code>	string	—	XRP Ledger WebSocket endpoint (wss://)
<code>xrp_required_ledger_closes</code>	uint32	1	Ledger closes for isFinal=true (deterministic)
<code>xrp_memo_type</code>	string	"URIB"	Memo type field for commitment embedding
<code>xrp_iou_circular_trust_check</code>	boolean	true	Enable circular trust line detection before IOU mapping

Hashing: The XRP Ledger uses SHA512/256 (the first 256 bits of SHA-512) for all ledger and transaction hashing, except for account IDs which use RIPEMD160(SHA256(pubkey)). The adapter must use SHA512/256 when verifying ledger-reported transaction hashes.

Commitment Mechanism: The commitmentDigest is embedded in a Payment or OfferCreate transaction's Memo array: `{ MemoType: hex("URIB"), MemoData: commitmentDigest_hex }`. This preserves the commitment on-ledger immutably.

Amount normalisation (XRP native, worked example):

```
nativeAmount = 1,000,000,000 drops (1,000 XRP) nativeDecimals = 6 C_AMT
(pre-FX) = 1,000,000,000 × 1012 = 1,000,000,000,000,000,000,000 (1021)
epochFxRate = fxNumerator: 50, fxDenominator: 100 (XRP = $0.50 in SDR)
C_AMT = floor(1021 × 50 / 100) = 500,000,000,000,000,000,000 URIB units
```

IOU Handling: XRP Issued Currency amounts (IOUs) represent balance claims against an issuer. Before mapping to C_AMT, the adapter must: (1) verify the trust line exists and is not frozen; (2) detect circular trust relationships (IOU of IOU of same issuer creates phantom value); (3) convert the IOU amount using the epoch FX rate for the IOU's currency code. Circular trust detection must traverse the full trust line graph for each IOU before normalisation.

Common Integration Pitfalls:

- **Circular trust lines:** XRP allows trust lines where A trusts B (USD) and B trusts A (USD), creating circular obligations that can inflate apparent value. Validate no circular trust exists before mapping IOU amounts.
- **OfferDelete invalidating paths:** Payment Paths in XRP are resolved at transaction execution time. An OfferDelete transaction concurrent with a Payment can cause path failures. Monitor for path failure result codes (tecPATH_PARTIAL, tecPATH_DRY) and treat them as mapping failures.
- **Destination tag requirements:** Many XRP accounts set the RequireDestTag flag. Payments to such accounts without a destination tag are rejected by the ledger. The adapter must check the account's flags before submitting settlement transactions.

```

// XRP normaliseTransaction pseudocode function
normaliseTransaction(nativeTx: XRPTransaction, railCtx: RailContext): CST {
const txHash: bytes32 = SHA512_256(serialise_canonical_XRP(nativeTx)) const
threadId: bytes32 = keccak256(abi.encode("xrp:mainnet", txHash,
railCtx.epochAnchor)) let amount: bigint, fee: bigint if
(nativeTx.TransactionType == "Payment") { if
(isXRPAmount(nativeTx.Amount)) { amount = BigInt(nativeTx.Amount) //
drops fee = BigInt(nativeTx.Fee) } else { // IOU: validate
trust line, detect circular trust, convert via FX
validateNoCircularTrust(nativeTx.Amount.issuer, nativeTx.Amount.currency,
railCtx) amount = convertIOUtoDropEquivalent(nativeTx.Amount,
railCtx.fxRates) fee = BigInt(nativeTx.Fee) } } const
canonicalAmount: C_AMT = normaliseAmount(amount, "drop") const
canonicalFee: C_AMT = normaliseAmount(fee, "drop") // Payment Path
handling: build CST chain (CST-M-004) let parentThreadId = null if
(nativeTx.Paths && nativeTx.Paths.length > 0) { parentThreadId =
buildPathChain(nativeTx.Paths, railCtx) } const sourceIdentity: ICT
= normaliseIdentity(nativeTx.Account) const destinationIdentity: ICT =
normaliseIdentity(nativeTx.Destination) const obligation: ObligationType =
nativeTx.Sequence in railCtx.escrowRegistry
? "ESCROW" : "TRANSFER" const semanticPayload: bytes = abi.encode(
nativeTx.Sequence, nativeTx.DestinationTag ?? 0, nativeTx.Paths ?
keccak256(JSON.stringify(nativeTx.Paths)) : bytes32(0) ) const
commitmentDigest: bytes32 = keccak256(abi.encode( threadId,
railCtx.epochAnchor, sourceIdentity, destinationIdentity,
canonicalAmount, canonicalFee, obligation, keccak256(semanticPayload) ))
const fv: FinalityVector = { railNamespace: "xrp:mainnet", confirmations:
1, requiredConfirmations: 1, finalityType: "deterministic",
estimatedFinalityMs: 4000, isFinal: true // XRP: deterministic after 1
ledger close } return { threadId, epochAnchor: railCtx.epochAnchor,
railNamespace: "xrp:mainnet", sourceIdentity, destinationIdentity,
canonicalAmount, canonicalFee, obligation, semanticPayload, timestamp:
now_ms(), finalityVector: fv, commitmentDigest, parentThreadId } }

```

7.3 ISO 20022 Rail Adapter

ISO 20022 is an international standard for financial messaging, providing a rich XML-based data model for payments, securities, trade finance, and foreign exchange. Within URIB, the ISO 20022 adapter covers SEPA (SCT and SCT Inst), SWIFT (GPI and standard), FedNow, and RTP payment messaging. Each of these networks uses ISO 20022 message types but may differ in transport protocol, business rules, and settlement timing.

Unlike blockchain-based rails, ISO 20022 does not have a native hash-based identity for senders and receivers. Identity is expressed via BIC (Bank Identifier Code), IBAN (International Bank Account Number), or LEI (Legal Entity Identifier). The ISO 20022 adapter maps these composite identifiers to ICTs using Template ICT-ISO20022.

Message Type	Description	Obligation Mapping
pacs.008	FI to FI customer credit transfer	TRANSFER
pacs.009	FI to FI financial institution credit transfer (cover)	TRANSFER (parentThreadId → pacs.008)
pacs.002	Payment status report (confirmation)	Updates existing CST FinalityVector; not a new CST
camt.054	Bank to customer debit/credit notification	Updates existing CST; not a new CST
pain.001	Customer credit transfer initiation	ESCROW (pending acceptance by bank)
camt.056 (return)	FI to FI payment cancellation request	Negative TRANSFER with parentThreadId → original pacs.008

Hashing: The XML message is first canonicalised using W3C Canonical XML 1.0 (c14n) before hashing. SHA256 of the canonical XML is the message hash and is embedded in `semanticPayload`. This ensures that whitespace, attribute ordering, and namespace prefix variations do not affect the hash.

Commitment Embedding: The commitmentDigest is embedded in the message's `SupplementaryData` block, which ISO 20022 reserves for extensibility. The field path is:

```
Document/FIToFICstmrCdtTrf/SplmtryData/Envlp/URIB_Commitment.
```

Common Integration Pitfalls:

- **Missing BIC codes:** Some pacs.008 messages in non-SWIFT networks may omit the debtor or creditor BIC. Reject such messages with E-MAP-003 rather than proceeding with incomplete identity binding.
- **DecimalPoint locale:** Some legacy systems encode amounts with comma as decimal separator (European locale). Always parse amounts as decimal strings using a locale-independent parser before bigint conversion.
- **R-transactions (returns):** camt.056 cancellations and pacs.004 returns represent negative value flows. They must be mapped as TRANSFER obligations with negative sign equivalence — specifically, a new CST with parentThreadId linking to the original, and a note in semanticPayload indicating RETURN type. The C_AMT of the return must match the original thread's C_AMT (minus any fees retained).
- **Settlement timing ambiguity:** SEPA SCT Inst achieves finality in <10 seconds; standard SEPA SCT is T+1; SWIFT can be T+0 to T+2. The adapter must configure the correct `finalityType` per messaging network and never assume all ISO 20022 messages have the same finality characteristics.

```
// ISO 20022 normaliseTransaction pseudocode function
normaliseTransaction(nativeTx: ISO20022_XMLMessage, railCtx: RailContext):
CST { // 1. Validate schema  const msgType: string =
detectMessageType(nativeTx) // "pacs.008", "pacs.009", etc.  if
(!XSD_validate(nativeTx, ISO20022_schemas[msgType])) {    throw new
MappingError("E-MAP-008", "Schema validation failed for: " + msgType)  }
// 2. Canonicalise and hash XML  const c14n_xml: string = W3C_c14n(nativeTx)
const xmlHash: bytes32 = SHA256(c14n_xml) // 3. Extract amount and
currency  const nativeAmt: string = extract(nativeTx, "//InstdAmt")  const
currency: string = extract(nativeTx, "//InstdAmt/@Ccy")  const amount:
bigint = parseDecimalStrict(nativeAmt) // locale-independent // 4.
Extract BIC and IBAN, validate  const debtorBIC: string = extract(nativeTx,
"//DbtrAgt/FinInstnId/BICFI")  const creditorBIC: string = extract(nativeTx,
"//CdtrAgt/FinInstnId/BICFI")  const debtorIBAN: string = extract(nativeTx,
"//Dbtr/Id/OrgId/AnyBIC") || extract(nativeTx,
"//DbtrAcct/Id/IBAN")  if (!validateBIC(debtorBIC) ||
!validateBIC(creditorBIC)) {    throw new MappingError("E-MAP-003", "Invalid
or missing BIC")  }  if (!validateIBAN(debtorIBAN)) {    throw new
MappingError("E-MAP-003", "Invalid IBAN")  } // 5. Normalise identities
const sourceIdentity: ICT = normaliseIdentity(debtorBIC + ":" +
debtorIBAN)  const destinationIdentity: ICT = normaliseIdentity(creditorBIC
+ ":" +
extract(nativeTx,
"//CdtrAcct/Id/IBAN")) // 6. Compute C_AMT (currency-aware FX conversion)
const nativeDecimals: number = ISO4217_decimals(currency)  const
canonicalAmount: C_AMT = normaliseAmountWithCurrency(amount, nativeDecimals,
currency, railCtx)  const canonicalFee: C_AMT = BigInt(0) // Fees
typically not in pacs.008 body // 7. Map obligation  const obligation:
ObligationType = msgType == "pain.001" ? "ESCROW" : "TRANSFER" // 8.
```

```

Semantic payload    const clearingRef: string = extract(nativeTx,
"/ClrSysRef") || ""    const semanticPayload: bytes = abi.encode(xmlHash,
msgType.padEnd(8), debtorBIC, creditorBIC, clearingRef)    const txHash:
bytes32 = SHA256(c14n_xml)    const threadId: bytes32 =
keccak256(abi.encode("iso20022:sepa", txHash, railCtx.epochAnchor))    const
commitmentDigest: bytes32 = keccak256(abi.encode(    threadId,
railCtx.epochAnchor, sourceIdentity, destinationIdentity,
canonicalAmount, canonicalFee, obligation, keccak256(semanticPayload)    ))
const fv: FinalityVector = buildISOFinalityVector(msgType, railCtx.network)
return { threadId, epochAnchor: railCtx.epochAnchor, railNamespace:
railCtx.namespace,    sourceIdentity, destinationIdentity, canonicalAmount,
canonicalFee,    obligation, semanticPayload, timestamp: now_ms(),
finalityVector: fv,    commitmentDigest, parentThreadId: null } }

```

7.4 CBDC Rail Adapter

Central Bank Digital Currencies (CBDCs) represent a rapidly evolving category of payment rail. URIB supports both retail CBDCs (direct consumer-facing digital currency) and wholesale CBDCs (interbank settlement instruments). The architecture varies significantly across CBDC platforms: some use account-based models similar to commercial bank accounts; others use token-based models similar to blockchain UTXOs; and wholesale platforms like Project mBridge use a multi-ledger distributed architecture.

The defining characteristic of CBDC rails from a URIB integration perspective is mandatory regulatory compliance: every CBDC CST must carry a KYC/AML attestation hash, and identity ICTs must carry a compliance tier. These requirements are enforced by INV-CBDC-001 through INV-CBDC-003 and cannot be waived by adapter configuration.

Configuration Parameter	Type	Default	Description
<code>cbdc_platform_api_url</code>	string	—	CBDC platform REST/RPC endpoint
<code>cbdc_kyc_registry_url</code>	string	—	National KYC registry endpoint for attestation retrieval
<code>cbdc_attestation_validity_window_s</code>	uint32	86400	Maximum age (seconds) of KYC attestation before refresh required
<code>cbdc_tier0_max_per_tx</code>	C_AMT	50 CBDC units	Maximum per-transaction C_AMT for Tier 0 identities
<code>cbdc_mint_auth_required</code>	boolean	true	Require governance authorisation hash for MINT obligations

Programmable Money and Conditional Transfers: Many CBDC platforms support programmable conditions on transfers (time-locked, trigger-based, or multi-party). These map to ESCROW obligations in URIB. The programmability condition is encoded as a hash of the condition specification in `semanticPayload`. When the condition is fulfilled, the ESCROW CST is promoted to TRANSFER by a follow-up normalisation of the fulfilment transaction.

mBridge PvP (Payment vs. Payment): Project mBridge enables simultaneous multi-CBDC exchanges across central banks. PvP settlements map to ATOMIC obligations in URIB, with each CBDC leg producing a separate CST and the two-phase commit protocol coordinating simultaneous release across jurisdictions.

Common Integration Pitfalls:

- **Attestation expiry:** KYC/AML attestation hashes from national registries carry a validity timestamp. Adapters must check attestation expiry before each epoch close. An attestation that expires during an epoch's ACCUMULATING phase will cause INV-CBDC-001 violations for all CSTs produced after expiry.
- **Platform heterogeneity:** Do not assume uniform API structure, decimal precision, or programmability model across CBDC platforms. Each CBDC namespace entry in the registry specifies its platform's characteristics. Adapters must be parameterised per platform.
- **Regulatory jurisdiction boundaries:** Cross-border CBDC transfers may require dual-jurisdiction regulatory assertions (both sending and receiving country KYC). Adapters for cross-border CBDC rails must collect and commit both attestation hashes in `semanticPayload`.

```

// CBDC normaliseTransaction pseudocode function
normaliseTransaction(nativeTx: CBDCTransaction, railCtx: RailContext): CST {
// 1. Retrieve KYC attestation for source identity  const riaSource =
fetchRIA(nativeTx.senderAccountId, railCtx.kycRegistryUrl)  if
(riaSource.expiryTimestamp < railCtx.epochOpenTimestamp) {      throw new
MappingError("E-INV-CBDC-001", "KYC attestation expired for sender")  }
const riaHash: bytes32 = keccak256(abi.encode(riaSource))  // 2. Resolve
ICTs with tier information  const sourceIdentity:      ICT =
normaliseIdentity(nativeTx.senderAccountId)  const destinationIdentity: ICT
= normaliseIdentity(nativeTx.receiverAccountId)  // 3. Enforce tier limits
const sourceTier: uint8 = ICL_Registry.lookup(sourceIdentity).complianceTier
const canonicalAmount: C_AMT = normaliseAmount(nativeTx.amount,
nativeTx.decimals)  if (sourceTier == 0 && canonicalAmount >
railCtx.config.cbdc_tier0_max_per_tx) {      throw new MappingError("E-INV-
CBDC-002", "Tier 0 per-transaction limit exceeded")  }  // 4. Classify
obligation (conditional → ESCROW, plain → TRANSFER)  const obligation:
ObligationType = nativeTx.condition ? "ESCROW" : "TRANSFER"  // 5. Build
semantic payload with compliance data  const conditionHash: bytes32 =
nativeTx.condition      ? keccak256(nativeTx.condition)      : bytes32(0)
const semanticPayload: bytes = abi.encode(      riaHash,
riaSource.expiryTimestamp, sourceTier, railCtx.regulatoryDomain,
conditionHash  )  const txHash: bytes32 = SHA256(nativeTx.platformTxId)
const threadId: bytes32 = keccak256(abi.encode(railCtx.namespace, txHash,
railCtx.epochAnchor))  const canonicalFee: C_AMT =
normaliseAmount(nativeTx.fee ?? 0n, nativeTx.decimals)  const
commitmentDigest: bytes32 = keccak256(abi.encode(      threadId,
railCtx.epochAnchor, sourceIdentity, destinationIdentity,
canonicalAmount, canonicalFee, obligation, keccak256(semanticPayload)  ))
const fv: FinalityVector = {      railNamespace: railCtx.namespace,
confirmations: 1, requiredConfirmations: 1,      finalityType: "instant",
estimatedFinalityMs: 5000, isFinal: true  }  return { threadId,
epochAnchor: railCtx.epochAnchor, railNamespace: railCtx.namespace,
sourceIdentity, destinationIdentity, canonicalAmount, canonicalFee,
obligation, semanticPayload, timestamp: now_ms(), finalityVector: fv,
commitmentDigest, parentThreadId: null } }

```

7.5 EVM Rail Adapter (Ethereum & L2s)

EVM-compatible chains share the Ethereum Virtual Machine execution environment and use Keccak256 for all hashing operations. The adapter covers Ethereum mainnet (chain ID 1) and all EVM-compatible Layer 2 networks. L2s present a specific challenge: they have fast soft-finality (seconds to minutes) but require L1 state root finalization before URIB considers them cross-rail final. An L2 transaction is not marked `isFinal: true` until the corresponding L1 state root is confirmed on Ethereum mainnet.

The EVM adapter handles both native ETH transfers and ERC-20 token transfers. ERC-20 transfers require decimal lookup from the token contract (`decimals()` view function) and are not universally 18-decimal. The adapter must cache token decimal values per epoch to avoid repeated RPC calls.

Configuration Parameter	Type	Default	Description
<code>evm_rpc_url</code>	string	—	EVM JSON-RPC endpoint (per chain)
<code>evm_chain_id</code>	uint32	1	EIP-155 chain ID for replay protection
<code>evm_required_confirmations</code>	uint32	12	Block confirmations for <code>isFinal=true</code> (L1). L2: determined by L1 state root.
<code>evm_bridge_contract_address</code>	bytes20	—	Address of the URIB bridge contract on this chain
<code>evm_l2_l1_state_root_contract</code>	bytes20 null	null	L1 contract address for L2 state root finalization (null for L1 chains)

Commitment Mechanism: The URIB bridge contract emits a `CrossRailCommitment` event upon settlement instruction. Event signature: `CrossRailCommitment(bytes32 indexed threadId, bytes32 commitmentDigest, uint64 epochId)`. The event log entry serves as the RailAnchor. On-chain event anchoring provides immutable, publicly verifiable proof of commitment.

EIP-1559 Fee Handling: Since EIP-1559, Ethereum transactions carry `maxFeePerGas` and `maxPriorityFeePerGas`. The actual fee paid is $(\text{baseFee} + \text{priorityFee}) \times \text{gasUsed}$. The adapter must retrieve the transaction receipt (post-execution) to get actual `gasUsed` and compute actual fee in wei, then convert to C_AMT.

EIP-4844 Blob Transactions: Type 3 (blob-carrying) transactions are used for L2 data posting to L1. When processing blob transactions, the `commitmentDigest` must be anchored in calldata (not in blob data), as blobs are prunable and do not form a permanent on-chain record.

Common Integration Pitfalls:

- **L2 finality before L1:** Never set `isFinal: true` for an L2 CST before the L2's state root is finalized on L1. L2 sequencer-level confirmation is not cross-rail final.
- **Chain fork handling:** Track the canonical chain per chainId using the highest total difficulty (PoW) or justified finality (PoS with Gasper). Non-canonical chain transactions must not produce CSTs.
- **ERC-20 decimal assumption:** Do not assume all ERC-20 tokens are 18-decimal. Always call `decimals()` on the token contract. USDC is 6-decimal; WBTC is 8-decimal.
- **Reentrancy in bridge contracts:** Bridge contracts that accept external calls must implement reentrancy guards. A reentrant commitment call could produce duplicate threadIds (INV-U-006 violation).

```
// EVM normaliseTransaction pseudocode function
normaliseTransaction(nativeTx: EVMTransaction, receipt: EVMReceipt, railCtx:
RailContext): CST { // 1. Verify chain ID matches configured namespace if
(nativeTx.chainId != railCtx.config.evm_chain_id) { throw new
MappingError("E-MAP-001", "Chain ID mismatch") } // 2. Compute
transaction hash const txHash: bytes32 = Keccak256(RLP_encode(nativeTx))
// 3. Determine if ETH transfer or ERC-20 transfer let amount: bigint,
tokenAddress: bytes20 | null = null, tokenDecimals: uint8 = 18 if
(nativeTx.to == railCtx.config.evm_bridge_contract_address &&
receipt.logs.length > 0) { // ERC-20 transfer via bridge contract
const transferEvent = parseERC20TransferEvent(receipt.logs) tokenAddress
= transferEvent.tokenAddress tokenDecimals =
getTokenDecimals(tokenAddress, railCtx.cache) // cached per epoch amount
= transferEvent.amount } else { // Native ETH transfer amount =
nativeTx.value } // 4. Compute actual fee from receipt const
actualFeePaid: bigint = BigInt(receipt.gasUsed) *
(BigInt(receipt.effectiveGasPrice)) // baseFee + priorityFee const
canonicalFee: C_AMT = normaliseAmount(actualFeePaid, 18) // wei, 18 decimals
// 5. Normalise amount const canonicalAmount: C_AMT =
normaliseAmount(amount, tokenDecimals) // 6. Normalise identities const
sourceIdentity: ICT = normaliseIdentity(nativeTx.from) const
destinationIdentity: ICT = normaliseIdentity(nativeTx.to) // 7. Classify
obligation const obligation: ObligationType =
classifyEVMObligation(nativeTx, receipt) // 8. Semantic payload const
calldataHash: bytes32 = Keccak256(nativeTx.input) const abiCommitment:
bytes32 = tokenAddress ? getContractABICommitment(tokenAddress,
railCtx.cache) : bytes32(0) const semanticPayload: bytes = abi.encode(
nativeTx.nonce, nativeTx.to, calldataHash, abiCommitment,
railCtx.config.evm_chain_id ) // 9. Compute thread ID const threadId:
bytes32 = Keccak256(abi.encode(railCtx.namespace, txHash,
railCtx.epochAnchor)) const commitmentDigest: bytes32 =
Keccak256(abi.encode( threadId, railCtx.epochAnchor, sourceIdentity,
destinationIdentity, canonicalAmount, canonicalFee, obligation,
Keccak256(semanticPayload) )) // 10. Build finality vector - check if L2
requires L1 state root const isL2: boolean =
railCtx.config.evm_l2_l1_state_root_contract != null const fv:
FinalityVector = isL2 ? buildL2FinalityVector(txHash, railCtx) //
isFinal only after L1 state root confirmed :
```

```
buildL1FinalityVector(receipt.blockNumber, railCtx)    return { threadId,
epochAnchor: railCtx.epochAnchor, railNamespace: railCtx.namespace,
sourceIdentity, destinationIdentity, canonicalAmount, canonicalFee,
obligation, semanticPayload, timestamp: now_ms(), finalityVector: fv,
commitmentDigest, parentThreadId: null } }
```

7.6 Stellar Rail Adapter

Stellar is an account-based blockchain with native support for path payments (multi-hop multi-currency exchanges), Claimable Balances (escrow-like structures), muxed accounts (sub-accounts within a single G-address), and Soroban smart contracts. The Stellar Consensus Protocol (SCP) achieves deterministic finality within approximately 5 seconds.

Stellar transaction amounts are encoded in XDR (External Data Representation) format, and transaction hashes are SHA256 of the XDR-encoded transaction envelope. Stellar asset amounts are stored as 64-bit signed integers in stroops (1 XLM = 10,000,000 stroops; 7 decimal places). The adapter must handle both XLM native and Stellar-issued assets (ICs) including the Stellar DEX.

Configuration Parameter	Type	Default	Description
<code>stellar_horizon_url</code>	string	—	Stellar Horizon API endpoint
<code>stellar_rpc_url</code>	string	—	Stellar RPC endpoint (for Soroban)
<code>stellar_required_ledger_closes</code>	uint32	1	Ledger closes for isFinal=true
<code>stellar_manage_data_key</code>	string	"URIB_COMMITMENT"	MANAGE_DATA key for commitment embedding

Muxed Accounts: Stellar M-addresses encode a G-address and a 64-bit sub-ID. Each unique (G-address, sub-ID) pair maps to a child ICT with the G-address ICT as parent. This enables a single institutional key to represent multiple sub-accounts within URIB while maintaining traceability to the parent identity.

Claimable Balances: Stellar Claimable Balances lock value that can be claimed by a set of claimants satisfying conditions (time-based or unconditional). These map directly to ESCROW obligations. The Claimable Balance ID serves as part of the threadId computation instead of a transaction hash.

Soroban Smart Contracts: Stellar's smart contract platform emits contract events analogous to EVM event logs. URIB commitments on Soroban contracts are anchored via contract events in the same manner as EVM.

Common Integration Pitfalls:

- **Sequence number monotonicity:** Stellar accounts have a sequence number that must increase by exactly 1 per submitted transaction. The adapter must track confirmed sequence numbers per account and reject CSTs for transactions with unexpected sequence gaps, which may indicate failed or dropped transactions confusing the CST chain.
- **Path payment partial fill:** Stellar path payments can partially fill. The adapter must check the `send_amount` vs. `source_amount` in the transaction result and only map the actual settled amount to `C_AMT`.
- **Issued asset decimal precision:** Unlike XLM (7 decimals), Stellar-issued assets may be configured with different decimal representations depending on issuer practice. Verify the asset's canonical decimal representation against the issuer's account data before normalising.

```
// Stellar normaliseTransaction pseudocode function
normaliseTransaction(nativeTx: StellarTransactionEnvelope, railCtx:
RailContext): CST { // 1. Hash XDR transaction envelope const txXDR:
bytes = XDR_encode(nativeTx) const txHash: bytes32 = SHA256(txXDR) const
threadId: bytes32 = Keccak256(abi.encode("stellar:mainnet", txHash,
railCtx.epochAnchor)) // 2. Extract operation (assume Payment or
PathPayment for primary case) const op = nativeTx.tx.operations[0] let
amount: bigint, assetCode: string, assetIssuer: string if (op.type ==
"payment" || op.type == "pathPaymentStrictReceive") { amount =
BigInt(op.amount) // in stroops assetCode = op.asset.code
assetIssuer = op.asset.issuer ?? "native" } else if (op.type ==
"createClaimableBalance") { amount = BigInt(op.amount) assetCode
= op.asset.code assetIssuer = op.asset.issuer ?? "native" //
Claimable Balance → ESCROW; use balance ID as additional thread anchor }
// 3. Normalise amounts (stroops, 7 decimals) const canonicalAmount: C_AMT
= normaliseAmount(amount, 7) const feeStroops: bigint =
BigInt(nativeTx.tx.fee) const canonicalFee: C_AMT =
normaliseAmount(feeStroops, 7) // 4. Handle muxed accounts const
sourceAddr: string = nativeTx.tx.sourceAccount const destAddr: string =
op.destination const sourceIdentity: ICT =
normaliseIdentity(sourceAddr) // handles M-address const
destinationIdentity: ICT = normaliseIdentity(destAddr) // 5. Semantic
payload (CST-M-005 Stellar) const seqNum: uint64 =
BigInt(nativeTx.tx.seqNum) const muxSubId: uint64 | null =
isMuxed(sourceAddr) ? extractSubId(sourceAddr) : null const
semanticPayload: bytes = abi.encode( seqNum, muxSubId ?? 0n,
encodeOperationType(op.type), assetCode.padEnd(12).slice(0,12),
assetIssuer == "native" ? bytes32(0) : Keccak256(assetIssuer) ) const
obligation: ObligationType = op.type == "createClaimableBalance" ?
"ESCROW" : "TRANSFER" const commitmentDigest: bytes32 =
Keccak256(abi.encode( threadId, railCtx.epochAnchor, sourceIdentity,
destinationIdentity, canonicalAmount, canonicalFee, obligation,
Keccak256(semanticPayload) )) const fv: FinalityVector = {
railNamespace: "stellar:mainnet", confirmations: 1, requiredConfirmations: 1,
finalityType: "deterministic", estimatedFinalityMs: 5000, isFinal: true }
return { threadId, epochAnchor: railCtx.epochAnchor, railNamespace:
"stellar:mainnet", sourceIdentity, destinationIdentity, canonicalAmount,
canonicalFee, obligation, semanticPayload, timestamp: now_ms(),
finalityVector: fv, commitmentDigest, parentThreadId: null } }
```

7.7 Generic Rail Adapter Template (Emerging and Custom Rails)

The following step-by-step template enables integration teams to onboard a new rail not listed in Sections 7.1–7.6. All steps are mandatory. Steps may be completed in parallel where noted but all must be completed before adapter submission to the URIB Review Board.

18. **Register RailNamespace:** Choose a namespace string following the format `<rail-family>[:chain-id][:network]`. Submit a namespace registration request including: proposed namespace string, existing namespace conflicts check, rail description, and technical contact. Namespace registration is a prerequisite for all subsequent steps.
19. **Define Hash Function and Commitment Scheme:** Identify the hash function used by the rail for transaction hashing. Select a Layer 0 Math Primitive that matches. Define the commitment scheme (hash-based or Pedersen). Document the commitment embedding mechanism (how `commitmentDigest` appears on the rail's ledger).
20. **Implement Amount Normalisation:** Determine the native denomination and decimal places. Implement `normaliseAmount()` using the `C_AMT` formula (Section 2.3). Write test vectors for: minimum native unit, maximum representable amount, boundary cases, and amounts that would overflow before floor division.
21. **Implement Identity Mapping → ICT Template:** Define how the rail's native address format is converted to a canonical public key or unique hash. Document the ICT derivation path. Register the ICT template with the ICL (parallel with Steps 5–6).
22. **Implement Finality Watcher:** Implement `queryFinality()` using either polling or event-driven strategy. Set the correct `finalityType` (probabilistic / deterministic / instant). Document the basis for the `requiredConfirmations` value with citations from the rail's consensus documentation.

23. Run Invariant Compliance Test Suite: Execute all 10 mandatory test cases (see below) against the adapter implementation. All tests must pass before proceeding.

Conservation invariant (INV-U-001): multi-input/multi-output transaction.

Identity binding (INV-U-002): unknown address → E-MAP-003.

Epoch anchoring (INV-U-003): wrong epoch anchor → rejection.

Commitment irreversibility (INV-U-004): attempt to retract CRC → rejection.

Finality monotonicity (INV-U-005): simulated reorg → Reorg Event emitted.

Non-duplication (INV-U-006): duplicate threadId → E-MAP-010.

C_AMT precision (INV-U-007): boundary rounding → floor confirmed.

Semantic payload integrity (INV-U-008): payload tampered after digest → rejection.

Amount overflow (E-MAP-002): maximum uint256 × conversion factor → overflow detected.

Idempotency: same input twice → identical CST output.

24. Submit for URIB Review Board Approval: Package the adapter manifest (namespace registration, tech spec, test results, code repository link), test vector document, and security assessment. The Review Board conducts a technical audit (2–4 weeks), testnet integration period (4 weeks), and, upon approval, promotes the namespace to the official registry.

Section 8 — Worked Examples

8.1 Example: Bitcoin → Ethereum ERC-20 Atomic Swap

Scenario: Alice sends 0.5 BTC on Bitcoin mainnet; Bob receives 12,500 USDC on Ethereum mainnet.

STEP 1 — Bitcoin UTXO Selection and OP_RETURN Commitment

Alice UTXO input: txid:

0xa1b2c3d4e5f60718293a4b5c6d7e8f900102030405060708090a0b0c0d0e0f10

vout: 0 value: 50,000,000 satoshis (0.5 BTC) type: P2TR (Taproot, key-path)

Bitcoin transaction outputs: Output 0: P2TR to Bob's exchange deposit address, 49,900,000 sat (locked by HTLC) Output 1: OP_RETURN URIB0x...

(commitmentDigest placeholder, 36 bytes) Output 2: P2TR change to Alice, 50,000 sat (remainder after fee) Miner fee: 50,000 satoshis = 0.0005 BTC BTC

OP_RETURN data: 0x55524942 || commitmentDigest_BTC


```
keccak256(abi.encode( threadId_BTC, epochAnchor, sourceIdentity_BTC,  
destinationIdentity_BTC, canonicalAmount_BTC, canonicalFee_BTC, "ESCROW",  
keccak256(semanticPayload_BTC) )) =  
0xBTC_COMMITMENT_DIGEST_000000000000000000000000000000000001  
CST_BTC = { threadId:  
0xBTC_THREAD_ID_000000000000000000000000000000000000000001,  
epochAnchor:      0xEPOCH_ANCHOR_HASH_PLACEHOLDER_..._000001,  
railNamespace:    "btc:mainnet", sourceIdentity:   0xALICE_ICT_BTC_...001,  
destinationIdentity: 0xBOB_ICT_BTC_...001, canonicalAmount:  
47,405,000,000,000,000,000,000, canonicalFee:  
47,500,000,000,000,000,000, obligation:          "ESCROW", semanticPayload:  
[encoded bytes], finalityVector:    { confirmations: 0, requiredConfirmations: 6,  
finalityType: "probabilistic", isFinal: false }, commitmentDigest:  
0xBTC_COMMITMENT_DIGEST_...001, parentThreadId:   null }
```

STEP 3 — Ethereum USDC Transfer and Bridge Contract Event

EVM transaction: from: Bob's Ethereum wallet
(0xBob000000000000000000000000000000000001) to: URIB Bridge
Contract (0xBRIDGE000000000000000000000000000000000001) value: 0 ETH
(ERC-20 transfer) input: bridge.transferWithCommitment(
token=USDC_ADDRESS, to=0xAlice_ETH_addr,
amount=12500_000000 (12,500 USDC, 6 decimals),
threadRef=0xBTC_THREAD_ID_...001) gasLimit: 150000 gasUsed:
85000 baseFee: 10 gwei, priorityFee: 1 gwei actualFee = 85000 × 11 gwei =
935,000 gwei = 935,000,000,000,000 wei Event emitted: CrossRailCommitment(
threadId=0xETH_THREAD_ID_...,
commitmentDigest=0xETH_COMMITMENT_DIGEST_...,
epochId=current_epoch_id)

STEP 4 — ETH CST Construction

```
txHash_ETH = Keccak256(RLP_encode(evm_tx))      =  
0xETH_TX_HASH_0000000000000000000000000000000000000000000000000000000000000001  
threadId_ETH = Keccak256(abi.encode("evm:1:mainnet", txHash_ETH,  
epochAnchor))      =  
0xETH_THREAD_ID_0000000000000000000000000000000000000000000000000000000000000001  
canonicalAmount_ETH (USDC): nativeAmount = 12,500,000,000 (12,500 USDC  
at 6 decimals) C_AMT = floor(12,500,000,000 × 1012 ×  
fxNumerator/fxDenominator) (USDC ≈ 1 USD; FX rate: 1.0 USD per URIB SDR  
unit at this epoch) C_AMT = 12,500,000,000,000,000,000,000,000 URIB units //  
Matches BTC side's C_AMT (approx) → conservation check passes at SEP level  
canonicalFee_ETH: fee = 935,000,000,000,000 wei C_AMT =  
floor(935,000,000,000,000 × 100 × fxRate_ETH) = [ETH FX converted] obligation:  
ATOMIC (two-rail exchange, ATOMIC per CST-M-003) CST_ETH = { threadId:  
0xETH_THREAD_ID_...001, ...all fields populated analogously to CST_BTC...  
obligation: "ATOMIC", parentThreadId: null (linked via Obligation Graph, not  
parentThreadId) }
```

STEP 5 — Two-Phase Commit Protocol Execution

Phase 1 — Prepare: Orchestrator → BTC Adapter: PREPARE(threadId_BTC, htlcPreimageHash) BTC Adapter: HTLC already on-chain (Alice locked funds in Step 1) Prepare commit hash: keccak256("btc:mainnet" || htlcAmt || htlcExpiry || preimageHash) =
0xBTC_PREPARE_COMMIT_HASH_00
Return: PREPARE_OK Orchestrator → ETH Adapter: PREPARE(threadId_ETH, preimageHash) ETH Adapter: Lock 12,500 USDC in bridge escrow contract with preimage condition Prepare commit hash: keccak256("evm:1:mainnet" || usdcAmt || escrowExpiry || preimageHash) =
0xETH_PREPARE_COMMIT_HASH_00
Return: PREPARE_OK Both prepare commits received within 180s timeout: →

Phase 2 COMMIT Phase 2 — Commit: Orchestrator reveals preimage to both adapters simultaneously. BTC Adapter: HTLC preimage revealed on-chain → Bob claims HTLC output (0.499 BTC) ETH Adapter: Bridge contract releases 12,500 USDC to Alice's Ethereum address Both RailAnchors collected.

STEP 6 — SEP Construction and Final State

```
SEP = {  sepId: keccak256(abi.encode(epochAnchor, [threadId_BTC,
threadId_ETH], obligationHash)),  epochAnchor: 0xEPOCH_ANCHOR_...,
threads: [0xBTC_THREAD_ID_...001, 0xETH_THREAD_ID_...001],
totalCanonicalValue: canonicalAmount_BTC + canonicalAmount_ETH,  //
Obligation: ATOMIC swap valued at ~$47,500 equivalent on both sides
obligationHash: keccak256(abi.encode(obligationGraphNode_AtomicSwap_1)),
railAnchors: [  { railNamespace: "btc:mainnet", txId: 0xa1b2c3...001a, blockHeight:
854_210 },  { railNamespace: "evm:1:mainnet", txId: 0xETH_TX_HASH_...001,
blockHeight: 20_000_001 }  ],  aggregateSignature: [64-byte Schnorr threshold sig
from 3-of-5 validators],  finalityVectors: [  { railNamespace: "btc:mainnet",
confirmations: 6, isFinal: true },  { railNamespace: "evm:1:mainnet", confirmations:
12, isFinal: true }  ],  settlementTimestamp: 1785218400000,  // Unix ms, ~July
2026  status: "COMPLETE" }  EMIT: SETTLED event { sepId, obligationId:
"AtomicSwap_1", epochId: current_epoch }
```

8.2 Example: XRP → SEPA ISO 2002 Payment

Scenario: Carol sends 1,000 XRP from XRP Ledger to a SEPA bank account via ISO 20022 pacs.008.

STEP 1 — XRP Payment Transaction → CST

XRP Transaction: Account: rCarolXRPAddress0000000000000001 (Carol's XRP account) Destination: rIntermediaryGateway000000000001 (SEPA gateway account) Amount: 1,000,000,000 drops (1,000 XRP) Fee: 12 drops Sequence: 42001 Memos: [{ MemoType: hex("URIB"), MemoData: commitmentDigest_XRP_hex }] txHash_XRP = SHA512_256(canonical_XRP_tx_serialisation) = 0xXRP_TX_HASH_00 threadId_XRP = keccak256(abi.encode("xrp:mainnet", txHash_XRP, epochAnchor)) = 0xXRP_THREAD_ID_00 canonicalAmount_XRP: $1,000,000,000 \text{ drops} \times 10^{12} \times \text{fxRate}(\text{XRP} \rightarrow \text{SDR}) = 500,000,000,000,000,000,000,000 \text{ URIB units (at XRP} = \0.50 SDR) CST_XRP = { threadId: 0xXRP_THREAD_ID_...001, railNamespace: "xrp:mainnet", sourceIdentity: ICT(rCarolXRPAddress...001), destinationIdentity: ICT(rIntermediaryGateway...001), canonicalAmount: 500_000_000_000_000_000_000, canonicalFee: $12 \times 10^{12} \times \text{fxRate}(\text{XRP} \rightarrow \text{SDR})$, obligation: "TRANSFER", finalityVector: { finalityType: "deterministic", confirmations: 1, isFinal: true }, ... }

STEP 2 — pacs.008 Generation and XML Hash

[illegible]

STEP 3 — Thread Chaining and ICT Resolution

Thread chain: CST_ISO has parentThreadId = threadId_XRP (ISO 20022 message is the correspondent settlement for the XRP payment) ICT Resolution:

Carol: XRP binding: ICT(rCarolXRPAAddress...001) → xrp:mainnet:rCarol...

ISO binding: ICT(DEUTDEDB:DE89370400440532013000) → iso20022:sepa:DEUTDEDB:... Cross-rail: Same individual; ICL registry links both bindings under one entity record (requires Carol to have registered both rail bindings with dual-sig proofs) BeneficiaryBank: ISO binding: ICT(BNPAFRPP:FR7630006000011234567890189) CST_ISO = { threadId: 0xISO_THREAD_ID_...001, railNamespace: "iso20022:sepa", sourceIdentity: ICT(DEUTDEDB:DE89...), destinationIdentity: ICT(BNPAFRPP:FR76...), canonicalAmount: $500_{00} \times 10^{16} \times \text{fxRate}(\text{EUR} \rightarrow \text{SDR})$, obligation: "TRANSFER", parentThreadId: 0xXRP_THREAD_ID_...001, // thread chain link finalityVector: { finalityType: "instant", confirmations: 1, isFinal: true }, ... }

STEP 4 — SEP with FinalityVectors from Both Rails

```
SEP = {  sepId: keccak256(abi.encode(epochAnchor, [threadId_XRP,
threadId_ISO], obligationHash)),  threads: [0xXRP_THREAD_ID_...001,
0xISO_THREAD_ID_...001],  totalCanonicalValue: sum(canonicalAmount_XRP,
canonicalAmount_ISO),  obligationHash:
keccak256(obligationNode_XRP_to_SEPA),  railAnchors: [  { railNamespace:
"xrp:mainnet",  txId: 0xXRP_TX_HASH_..., ledgerIndex: 91_234_567 },  {
railNamespace: "iso20022:sepa",  txId: 0xISO_XML_HASH_..., clearingRef: "URIB-
XRP-SEPA-20260708-001" }  ],  finalityVectors: [  { railNamespace:
"xrp:mainnet",  isFinal: true, finalityType: "deterministic" },  { railNamespace:
"iso20022:sepa", isFinal: true, finalityType: "instant" }  ],  aggregateSignature: [3-
of-5 threshold Schnorr signature],  status: "COMPLETE" }
```

8.3 Example: CBDC Retail Transfer with Compliance Assertion

Scenario: David (Tier 1 verified individual) transfers 500 digital euros (ECB CBDC) to Emma (Tier 1 verified individual).

STEP 1 — KYC Attestation Retrieval

[illegible]

STEP 2 — ICT-CBDC Template Instantiation

[illegible]

STEP 3 — CST Construction with Compliance Fields

```
nativeAmount = 500_000_000_000_000_000_000 (500 digital euros, 18-decimal
CBDC) canonicalAmount_CBDC: nativeDecimals = 18 → conversion factor = 10^0
= 1 C_AMT = floor(500_000_000_000_000_000_000 × 1 × fxRate(EUR→SDR))
semanticPayload_CBDC = abi.encode(   riaHash_David,           // sender attestation
1767225600,           // attestation expiry 1,           // complianceTier
"ECB_EUR",           // regulatory domain (8 bytes) bytes32(0)           // no
programmability condition (plain transfer) ) CST_CBDC = {   threadId:
keccak256("cbdc:eur:ecb" || platfmTxHash || epochAnchor),   railNamespace:
"cbdc:eur:ecb",   sourceIdentity:   ICT_David,   destinationIdentity: ICT_Emma,
canonicalAmount:   [computed above],   canonicalFee:   0, // ECB CBDC: no
transaction fees for retail   obligation:   "TRANSFER",   semanticPayload:
[encoded bytes above],   finalityVector:   { finalityType: "instant",   confirmations: 1,
isFinal: true },   ... } Invariant checks: INV-U-001: sum(debits) = 500 dEUR =
sum(credits) + 0 fee ✓ INV-U-002: both ICTs verified in ICL ✓ INV-CBDC-001:
David's attestation expiry (2027-01-01) > epoch open (2026-07-08) ✓ INV-CBDC-
002: Tier 1 limit not exceeded (500 EUR < Tier 1 max) ✓ Single-rail settlement:
SEP produced with 1 thread, 1 RailAnchor.
```

8.4 Example: Stellar Path Payment Bridged to Lightning Network (BTC)

Scenario: Frank sends 200 USDC (Stellar issued asset) via Stellar path payment; Grace receives 0.005 BTC via Lightning Network HTLC.

STEP 1 — Stellar Path Payment → Multi-Hop CST Chain

Stellar PathPaymentStrictReceive: SourceAccount:
GFrankXLMAddress000000000000000000000000000000000001 Destination:
GLightningGateway000000000000000000000000000000000001 SendAsset:
USDC (issuer: GUSDC_ISSUER_STELLAR_000001) SendMax: 200.0000000
USDC = 2,000,000,000 stroops (7 decimal places) DestAsset: XLM (for
gateway conversion) DestAmount: [gateway quote] Path: [USDC] → [XLM] →
[BTC-pegged token] (3 hops) Hop 1 CST (USDC → XLM): threadId_hop1 =
keccak256("stellar:mainnet" || hop1_txHash || epochAnchor) canonicalAmount:
200 USDC → C_AMT via 7 decimals and FX rate obligation: TRANSFER
parentThreadId: null (root of chain) Hop 2 CST (XLM → BTC-peg):
threadId_hop2 = keccak256("stellar:mainnet" || hop2_txHash || epochAnchor)
parentThreadId: threadId_hop1 Hop 3 CST (BTC-peg → gateway): threadId_hop3
= keccak256("stellar:mainnet" || hop3_txHash || epochAnchor) parentThreadId:
threadId_hop2 Chain validation (CST-M-004): hop1 → hop2 → hop3 → Lightning
gateway → no cycle ✓ All three hops in same epoch ✓ Aggregate C_AMT
conservation across chain ✓

STEP 2 — Lightning Network HTLC → ESCROW CST

Lightning HTLC (BTC Lightning sub-adapter): HTLC amount: 500,000 satoshis (0.005 BTC) preimageHash: SHA256(preimage) = 0xPREIMAGE_HASH_00000000000000001 expiry: Grace's invoice expiry + 144 blocks (safety margin) channel: Frank_Gateway_Lightning_Channel_001

```
CST_Lightning = { threadId: keccak256("btc:mainnet" || HTLC_channel_id ||
htlc_id || epochAnchor), railNamespace: "btc:mainnet", sourceIdentity:
ICT_LightningGateway, destinationIdentity: ICT_Grace, canonicalAmount:
500,000 sat × 1010 × fxRate(BTC→SDR), obligation: "ESCROW", // HTLC →
ESCROW semanticPayload: abi.encode(ScriptType.HTLC, preimageHash,
htlcExpiry, 0, 2), parentThreadId: threadId_hop3 // linked to Stellar chain }
```

STEP 3 — Abort Scenario (One Hop Fails)

Abort Trigger: Scenario: Lightning HTLC times out (gateway cannot forward to Grace within invoice expiry). Finality watcher detects HTLC timeout output claimed by gateway (refund path). Rail Reorg Event NOT triggered; HTLC timeout is expected protocol behaviour. Abort Processing: Bridge Orchestrator receives ABORT signal from Lightning sub-adapter. CST_Lightning marked: obligation=ESCROW, status=ABORTED, finalityVector.isFinal=false. Orchestrator issues ABORT to Stellar adapter for the CST chain (hop1, hop2, hop3). Stellar path payment is NOT aborted on-chain (already completed on Stellar). Instead: a REVERSAL CST is created for the Stellar leg, parentThreadId → threadId_hop1, negative value flow, obligation → TRANSFER (return), C_AMT = original amount. SEP is NOT produced for this obligation. Obligation node marked: ABORTED. Retry or dispute resolution triggered. Frank is refunded via the Stellar reversal path.

8.5 Example: Multi-Rail Obligation Graph Settlement

Scenario: Corporate treasury settlement: EUR ISO 20022 pacs.009 cover + USDC on Polygon (EVM) + XRP as liquidity bridge. Three CSTs, one Obligation Graph node.

OBLIGATION GRAPH NODE

```
ObligationGraphNode { id:
"CORP_TREASURY_SETTLE_20260708_001", requiredValue:
50,000,000,000,000,000,000,000 URIB units (~$50,000 SDR) threads:
[threadId_ISO, threadId_POLY, threadId_XRP] status:    PENDING type:
ATOMIC // all three rails must settle simultaneously }
```

THREE CSTs CONTRIBUTING TO THE OBLIGATION

```
CST_ISO (pacs.009 cover, EUR, SWIFT): threadId:
0xISO_CORP_THREAD_ID_...001 railNamespace: "iso20022:swift"
canonicalAmount: 20,000 EUR × 1016 × fxRate(EUR→SDR) obligation:
TRANSFER finalityVector: { finalityType: "deterministic", isFinal: true } CST_POLY
(USDC on Polygon, evm:137): threadId:    0xPOLY_CORP_THREAD_ID_...001
railNamespace: "evm:137:polygon" canonicalAmount: 25,000 USDC × 1012 ×
fxRate(USDC→SDR) obligation:  TRANSFER finalityVector: { finalityType:
"probabilistic", isFinal: false } // Note: L2 — not final until L1 state root confirmed
on Ethereum mainnet // Adapter is polling L1 state root contract
(evm_l2_l1_state_root_contract) CST_XRP (XRP liquidity, xrp:mainnet): threadId:
```

0xXRP_CORP_THREAD_ID_...001 railNamespace: "xrp:mainnet"
canonicalAmount: 10,000 XRP drops-equivalent $\times$ fxRate(XRP $\rightarrow$ SDR) obligation:
TRANSFER finalityVector: { finalityType: "deterministic", isFinal: true }
Conservation check: $\text{sum}(\text{canonicalAmount_ISO} + \text{canonicalAmount_POLY} + \text{canonicalAmount_XRP}) == \text{obligationNode.requiredValue}$ ✓ (all three legs
together fulfill the \$50,000 obligation)

SEP CONSTRUCTION WITH THREE RAIL ANCHORS

```
SEP = { sepld: keccak256(abi.encode( epochAnchor, [threadId_ISO,
threadId_POLY, threadId_XRP],
keccak256(obligationGraphNode_CORP_TREASURY_...) )), threads:
[0xISO_CORP_THREAD_ID_...001, 0xPOLY_CORP_THREAD_ID_...001,
0xXRP_CORP_THREAD_ID_...001], totalCanonicalValue: sum of all three
C_AMTs == requiredValue ✓, obligationHash:
keccak256(abi.encode(obligationNode_CORP_TREASURY_SETTLE_20260708_0
01)), railAnchors: [ { railNamespace: "iso20022:swift", txId:
0xISO_XML_HASH_..., clearingRef: "SWIFT-GPI-..." }, { railNamespace:
"evm:137:polygon", txId: 0xPOLY_TX_HASH_..., blockHeight: 61_200_000,
l1StateRootTxId: 0xETH_STATE_ROOT_TX_... }, // L1 finalization anchor {
railNamespace: "xrp:mainnet", txId: 0xXRP_TX_HASH_..., ledgerIndex:
91_234_999 } ], finalityVectors: [ { railNamespace: "iso20022:swift", isFinal:
true, finalityType: "deterministic" }, { railNamespace: "evm:137:polygon", isFinal:
true, finalityType: "probabilistic", note: "L1 state root confirmed at block
20_000_050" }, { railNamespace: "xrp:mainnet", isFinal: true, finalityType:
"deterministic" } ], aggregateSignature: [48-byte BLS12-381 aggregate sig from
validators v1, v2, v3], // Threshold 3-of-5; validators v1, v2, v3 signed
settlementTimestamp: 1785222000000, status: "COMPLETE" } Obligation node
updated: status = SETTLED, sepld = [above] EMIT: SETTLED event { sepld,
obligationId: "CORP_TREASURY_SETTLE_20260708_001" }
```

Section 9 — Bridge Orchestrator & Epoch Management

9.1 Epoch Lifecycle Management

Transition	Trigger Condition	State Guard Checks	Events Emitted
→ OPEN	Previous epoch enters SETTLED; epoch timer starts	Previous epoch settlement root committed; validator set quorum present; FX oracle commitment fresh	EPOCH_OPEN(epochId, anchorHash, openTimestamp)
OPEN → ACCUMULATING	Immediate upon epoch OPEN	epochAnchor hash published and immutable	EPOCH_ACCUMULATING(epochId)
ACCUMULATING → CLOSING	epoch_close_window seconds before epoch end	All in-flight CSTs inventoried; carry-forward candidates identified	EPOCH_CLOSING(epochId, pendingThreadCount, carryForwardCount)
CLOSING → SETTLED	All SEPs complete OR finality grace period expires	All Obligation Graph nodes in SETTLED or ABORTED state; validator threshold sig obtained	EPOCH_SETTLED(epochId, settlementRoot, sepCount)
SETTLED → ARCHIVED	archive_delay seconds after SETTLED	Settlement root backed up to cold storage; all SEPs stored	EPOCH_ARCHIVED(epochId, settlementRoot)

9.2 Obligation Graph

The **Obligation Graph** is a directed acyclic graph (DAG) maintained by the Bridge Orchestrator. Each node represents a settlement obligation — a requirement that a specific canonical value is transferred from a source identity to a destination identity. Edges represent CST dependencies: edge $A \rightarrow B$ means CST A must be settled before obligation B can be resolved (e.g., a cover payment must settle before the customer credit transfer is confirmed).

Settlement ordering is determined by topological sort (Kahn's algorithm) over the Obligation Graph. Obligations with no incoming edges (root nodes) are settled first. Obligations with dependencies (non-root nodes) are settled after all their dependency edges are satisfied. Cycle detection is run at graph construction time; cycles are structural errors that indicate a CST chaining violation (E-MAP-005).

The Obligation Graph is rebuilt from scratch at each epoch OPEN, populated during ACCUMULATING, and consumed during CLOSING. The graph is stored in-memory during the epoch and its root hash is included in the settlement root at epoch SETTLED.

9.3 Validator Set & Threshold Signatures

The URIB validator set consists of a minimum of 5 independent validators operating in geographically distributed, operationally isolated environments. The default threshold is 3-of-5 (60% supermajority). Deployments may configure up to 15 validators with a threshold of up to 10-of-15.

Two aggregate signature schemes are supported:

- **Schnorr Threshold (secp256k1):** Produces a 64-byte aggregate signature indistinguishable from a standard Schnorr signature. Requires an interactive signing protocol (Musig2 or FROST). Used by default for EVM-compatible deployments where Schnorr verification is gas-efficient.
- **BLS12-381 Aggregate Signature:** Produces a 48-byte aggregate signature. Non-interactive: validators sign independently and signatures are aggregated offline. Preferred for high-throughput deployments where interactive rounds are impractical.

Validator rotation occurs at most once per epoch and requires a key ceremony: outgoing validator's key share is securely destroyed; incoming validator participates in a distributed key generation (DKG) ceremony with remaining validators; new aggregate public key is committed to the next epoch anchor. Rotation procedures must be completed during an ACCUMULATING state, never during CLOSING.

9.4 Dispute Resolution Engine

The Dispute Resolution Engine implements a three-tier escalation ladder. Each tier has defined SLAs and escalation triggers.

Tier	Resolver	SLA	Trigger for Escalation	Resolution Authority
Tier 1	Bridge Orchestrator (automated)	5 minutes	Cannot determine outcome cryptographically within 5 min; or outcome is contested by a validator	Automated logic; no human input
Tier 2	Validator Set Vote	2 hours	No validator supermajority within 2 hours; legal/regulatory dimension identified	m-of-n validator vote (default 3-of-5)
Tier 3	Human Governance Committee	5 business days	Tier 2 deadlock; dispute value exceeds governance threshold; cross-jurisdictional legal issue	URIB Governance Committee (human); binding decision recorded on-chain

9.5 Monitoring & Alerting

Metric	Description	Warning Threshold	Critical Threshold	On-Call Action
CST Throughput	CSTs admitted per second	<80% of baseline	<50% of baseline	Check adapter RPC connectivity; check CST Engine queue depth
SEP Success Rate	% of obligation nodes achieving COMPLETE SEP	<95%	<85%	Investigate ABORTED obligation nodes; check finality watcher health
Finality Latency per Rail	Time from CST admission to isFinal=true	>1.5× expected	>3× expected	Check rail node sync status; evaluate carry-forward necessity
Epoch Close Time	Time from CLOSING trigger to SETTLED	>90% of closing window	Epoch misses SETTLED deadline	Emergency carry-forward of all non-final threads; alert governance
Invariant Violation Rate	INV violations per 1,000 CSTs	>1 per 1,000	>10 per 1,000	Investigate specific adapter; consider adapter suspension
Open Disputes	Count of SEPs in DISPUTED state	>3 open	>10 open	Escalate to Tier 2; alert governance committee

Section 10 — Security Considerations

10.1 Threat Model

Assets under protection: (1) Canonical value in-flight (C_AMT in uncommitted CSTs); (2) Settled value committed in completed SEPs; (3) Identity bindings in the ICL registry; (4) Epoch state and Obligation Graph; (5) Validator private key shares.

Threat Actor	Capabilities	Primary Attack Surfaces	Mitigation
External attacker	Network access; no URIB credentials	Rail adapter RPC endpoints; CST Engine API; replay attacks	TLS/mTLS on all interfaces; threadId uniqueness (INV-U-006); epoch anchoring (INV-U-003)
Malicious rail adapter	Valid adapter credentials; can publish CRCs	Amount inflation (INV-U-007); duplicate threadIds (INV-U-006); synthetic identity binding (INV-U-002)	Adapter sandboxing; invariant validation pipeline; bond/slash mechanism
Compromised validator	One validator key share	SEP signature forgery; epoch anchor manipulation	m-of-n threshold (minimum 3-of-5); validator key rotation; validator isolation
Compromised rail node	Serves false transaction data to adapter	Finality falsification; reorg concealment	Multi-node RPC; finality monotonicity (INV-U-005); cross-rail corroboration for ATOMIC

10.2 Rail Adapter Sandboxing

Each rail adapter runs in an isolated execution context enforced at the process and network level.

Sandbox constraints:

- **Read-only rail access:** Adapters may only read from their designated rail node RPC endpoints. Write access (transaction submission) is permitted only for the `submitSettlement()` method and only for transactions constructed from SEP instructions received from the Bridge Orchestrator.
- **No direct CST Engine write access:** Adapters publish CRCs via an authenticated message queue. The CST Engine validates all CRCs against invariants before admission. Adapters cannot directly write to the CST Engine's state store.
- **Network isolation:** Adapter processes have egress rules permitting connections only to: their designated rail node endpoint, the URIB message queue, and the ICL resolver. No other outbound connections are permitted.
- **Resource limits:** Each adapter process is allocated fixed CPU and memory quotas. Unbounded computation (e.g., during circular trust detection) is subject to timeout.

10.3 Replay Attack Prevention

URIB implements two complementary replay prevention mechanisms:

- **threadId uniqueness (INV-U-006):** The threadId incorporates the epoch anchor, making it impossible to replay a CST from a previous epoch. Even if a native transaction ID is reused on a fork or reorganised chain, the threadId will differ due to the different epoch anchor.
- **Epoch anchoring (INV-U-003):** A CST committed to epoch N cannot be replayed in epoch N+1 because the epoch anchor hash has changed. An attacker attempting to replay a settled CST from a previous epoch will produce a threadId that already exists in the registry (INV-U-006 violation) or an epoch anchor mismatch (INV-U-003 rejection).

Additionally, a persistent nonce registry maintained by the CST Engine records all threadIds across all epochs (not just the current epoch) for the purpose of double-spend detection across epoch boundaries. This registry is append-only and persists indefinitely.

10.4 Economic Security

Every registered rail adapter and every validator must post a bond denominated in a stable reference asset (URIB governance token or designated stablecoin). The bond serves as an economic disincentive against malicious behaviour.

Slashing conditions and magnitudes:

Condition	Slash Magnitude	Recovery Path
INV-U-007 violation (synthetic value creation)	100% of adapter bond	None — permanent adapter suspension
INV-U-006 violation (deliberate duplicate threadId)	50% of adapter bond	Adapter re-certification required after 30-day suspension
Invalid SEP signature (compromised validator)	100% of validator bond	Key rotation ceremony; new validator onboarding required
Finality falsification (adapter reports isFinal=true prematurely)	25% of adapter bond	Finality watcher audit; bond restoration after audit
Unresponsive adapter (>3 consecutive epochs without activity)	10% of adapter bond	Restart and health check; bond restored on recovery

10.5 Cryptographic Agility

URIB is designed for cryptographic agility: the ability to migrate from current cryptographic primitives to post-quantum alternatives without a full system restart. The migration path for post-quantum upgrades:

25. **Algorithm registry versioning:** Every Math Primitive in Layer 0 carries a version tag. The CST schema includes a `cryptoVersion` field that identifies which Layer 0 algorithm set was used to produce the CST.
26. **Hash function migration:** When upgrading (e.g., from SHA256 to a hash-based PQ alternative like SPHINCS+), the new hash function is registered in Layer 0. Adapters emit CSTs with the new `cryptoVersion`. A transition period (minimum 4 epochs) allows both versions to coexist.
27. **Signature scheme migration:** ICT binding proofs include a signature scheme identifier. When migrating from secp256k1 to a lattice-based scheme (e.g., ML-DSA/CRYSTALS-Dilithium), existing ICT versions remain valid; new ICT versions use the new scheme. The migration window is 10 epochs.
28. **ZK proof upgrade:** Zero-knowledge gadgets (range proofs, hash preimage proofs) are modular. Each gadget is independently upgradeable. Proof scheme identifiers are included in SEP aggregateSignature encoding.

Section 11 — Deployment & Operations

11.1 Adapter Registration Process

29. **Submit Adapter Manifest:** Complete the adapter manifest template, including: proposed RailNamespace string, technical specification document, adapter software repository URL, test vector document, security assessment (self-conducted), and contact information for primary and secondary technical contacts.
30. **URIB Review Board Technical Audit (2–4 weeks):** The Review Board evaluates: correctness of invariant implementation, absence of synthetic value creation vulnerabilities, identity mapping security, finality watcher reliability, code quality and test coverage (>90% line coverage required), and dependency supply chain audit.
31. **Testnet Integration Period (4 weeks):** Adapter is deployed against URIB testnet infrastructure. Minimum requirements: 1,000 CSTs successfully processed; 100 SEPs produced; full adversarial test suite passed (reorg simulation, overflow attempts, identity spoofing).
32. **Mainnet Approval:** Review Board issues mainnet approval. Adapter submits bond. RailNamespace is added to the official registry with status ACTIVE.
33. **Post-Launch Monitoring Period (30 days):** Enhanced monitoring with daily invariant violation reports. Any REJECT-severity violation triggers immediate Review Board notification. HALT-severity violations trigger automatic suspension.

11.2 Configuration Reference

Parameter	Type	Default	Description
<code>epoch_duration_s</code>	uint32	600	Epoch duration in seconds. Range: [60, 3600].
<code>epoch_close_window_s</code>	uint32	60	Seconds before epoch end to enter CLOSING state. Must be < <code>epoch_duration_s</code> .
<code>finality_grace_period_ms</code>	uint64	30000	Additional ms after CLOSING before forcing carry-forward for non-final threads.
<code>fx_oracle_address</code>	string	—	Address or URL of the URIB FX Oracle service. Required.
<code>fx_staleness_threshold_s</code>	uint32	120	Max age (seconds) of FX oracle commitment before adapter throws E-MAP-011.
<code>validator_threshold_m</code>	uint8	3	Required validator signatures for SEP threshold (m in m-of-n).
<code>validator_count_n</code>	uint8	5	Total validators in the set (n in m-of-n). Must be ≥ 5.
<code>dispute_tier1_timeout_s</code>	uint32	300	Tier 1 automated dispute resolution timeout in seconds.
<code>dispute_tier2_timeout_s</code>	uint32	7200	Tier 2 validator vote dispute resolution timeout in seconds.
<code>max_threads_per_epoch</code>	uint64	1,000,000	Maximum CSTs admissible in a single epoch. Hard limit; additional threads rejected.
<code>max_c_amt_per_thread</code>	C_AMT (uint256)	2^200	Maximum C_AMT for a single thread. Prevents runaway values from misconfigurations.
<code>finality_staleness_threshold_ms</code>	uint64	120000	Max age (ms) of FinalityVector before thread is marked STALE.
<code>ict_migration_window_epochs</code>	uint32	2	Number of epochs old ICT version remains valid after key rotation.
<code>max_thread_chain_depth</code>	uint32	32	Maximum parentThreadId chain length (cycle detection upper bound).
<code>archive_delay_s</code>	uint32	604800	Seconds after SETTLED before epoch transitions to ARCHIVED (default: 7 days).

11.3 Testing Requirements

Every adapter must pass the following mandatory test categories before mainnet approval. Test results must be reproducible and submitted as part of the adapter manifest.

Unit Tests (per adapter)

- Invariant checks: all 8 universal invariants, each with positive (passes) and negative (fails as expected) test cases.
- Amount normalisation: test vectors for minimum unit, maximum representable value, boundary rounding (floor confirmed), zero-amount rejection, overflow detection.
- Identity mapping: known address → expected ICT (test vectors from template), unknown address → provisional ICT or E-MAP-003, key rotation → version increment.
- Semantic payload encoding: determinism (same input → same output), schema compliance per rail family.

Integration Tests (end-to-end, testnet)

- CST round-trip: native transaction → normalise → CST → buildCommitment → verifyCommitment → submitSettlement → queryFinality → isFinal=true.
- SEP generation: single-rail and two-rail SEP generation: single-rail and two-rail scenarios, conservation check, finality vector refresh.
- Thread chaining: multi-hop chain, cycle detection (must reject), carry-forward across epochs.

Adversarial Tests (mandatory for mainnet approval)

- **Reorg simulation:** Inject a simulated chain reorganisation; verify Reorg Event is emitted and affected CSTs are suspended within 30 seconds.
- **Duplicate thread injection:** Submit identical nativeTx twice; verify second submission is rejected with E-MAP-010 and no duplicate CST is created.
- **Amount overflow attempt:** Submit a nativeAmount of `type(uint256).max`; verify E-MAP-002 is thrown and no CST is created.

- **Identity spoofing attempt:** Submit a CST with a forged ICT that does not match the binding proof for the rail-native address; verify INV-U-002 violation is caught in Phase 2 validation.
- **Payload tampering:** Modify `semanticPayload` after computing `commitmentDigest`; verify INV-U-008 violation is caught.
- **Stale FX rate:** Advance the mock clock beyond `fx_staleness_threshold_s`; verify E-MAP-011 is thrown and no amount normalisation proceeds.

11.4 Upgrade & Versioning

Rail adapters use **Semantic Versioning** (semver): `MAJOR.MINOR.PATCH`. The `railVersion` field in the RAI contract must match the version registered in the RailNamespace Registry for the adapter to pass Phase 1 structural validation.

Version Increment	Trigger Condition	Backward Compatibility	Migration Required
PATCH (x.y.Z)	Bug fix; no interface change; no CST schema change	Full — no migration	No
MINOR (x.Y.0)	New optional field; new supported obligation type; new rail-specific invariant	Forward compatible — old CSTs remain valid	Optional (new features only)
MAJOR (X.0.0)	Breaking change: CST schema field change, <code>threadId</code> computation change, <code>commitmentDigest</code> algorithm change	None — breaking change	Yes — full migration procedure required

Breaking change policy: MAJOR version increments require a minimum backward compatibility window of 2 epochs during which both old and new adapter versions are simultaneously active. CSTs produced by old-version adapters remain valid and must be settled before the old version is deprecated. Adapter deprecation is announced a minimum of 14 days before the deprecation epoch. After the deprecation epoch, old-version CSTs are rejected by the CST Engine.

Adapter deprecation procedure:

34. URIB Review Board publishes deprecation notice with target epoch.
35. Adapter operator migrates all active integrations to new version.
36. In-flight CSTs from old version are allowed to complete (maximum 2-epoch carry-forward).
37. At deprecation epoch: old `railVersion` removed from registry; subsequent Phase 1 validation rejects old-version CSTs.
38. Old version namespace entry marked DEPRECATED in the RailNamespace Registry.

Appendices

Appendix A — Math Primitives Reference

HASH FUNCTIONS

Algorithm	Input	Output	Used By	Test Vector (input "abc")
SHA256	Arbitrary bytes	32 bytes	Bitcoin commitment, ISO 20022 XML hash, CBDC	ba7816bf8f01cfea414140de5dae2ec73b00361bbef0469207a...
SHA256d	Arbitrary bytes	32 bytes	Bitcoin txid computation	SHA256(SHA256("abc")) = 4f8b42c22dd3729b519ba6f68d2da7cc5b2d606d05daed5ad5128cc03e6c6358
SHA512/256	Arbitrary bytes	32 bytes (first 256 bits of SHA-512)	XRP Ledger transaction and ledger hashing	53048e2681941ef99b2e29b76b4c7dabe4c2d0c634fc6d46e0e2f13107e7af23
Keccak256	Arbitrary bytes	32 bytes	EVM txid, threadId computation, commitment Digest, ICT derivation	4e03657aea45a94fc7d47ba826c8d667c0d1e6e33a64a036ec44f58fa12d6c45

PEDERSEN COMMITMENT SCHEME

```
// Pedersen Commitment over secp256k1 or BLS12-381 // Setup: two independent
generators G and H (nothing-up-my-sleeve construction) // G = standard curve
generator // H = keccak256("URIB_PEDERSEN_H") × G (hash-to-curve)
Commit(value v, blinding r): C = v×G + r×H // C is a curve point; value v
and blinding factor r are field elements Open(C, v, r): ASSERT C == v×G +
r×H Homomorphic addition: Commit(v1, r1) + Commit(v2, r2) == Commit(v1+v2,
r1+r2) // Used for conservation verification without revealing individual
amounts Blinding factor management: r MUST be generated from a
cryptographically secure RNG. r is NOT stored by URIB; only the commitment
C and the opening hint (v) are stored. For hash-based commitments (non-
Pedersen rails): blinding factor is implicitly zero.
```

SCHNORR THRESHOLD SIGNATURE (FROST)

```
// FROST (Flexible Round-Optimised Schnorr Threshold) – 2-round interactive
protocol // Used for validator threshold signatures on SEPs Parameters: n
= validator count (default 5) t = threshold (default 3; t = m in m-of-n)
Group: secp256k1 Round 1 (Commitment): Each validator i generates nonce
pair (d_i, e_i) ← random scalars Computes commitment pair (D_i, E_i) =
(d_i×G, e_i×G) Broadcasts (D_i, E_i) to all participants Round 2
(Signing): Coordinator aggregates binding factor: ρ_i = H(i, msg, {(D_j,
E_j)}) Each validator i computes: R_i = D_i + ρ_i × E_i R = Σ R_i
(aggregate nonce) c = H(R, aggregatePubKey, msg) (challenge) z_i =
d_i + ρ_i×e_i + λ_i×s_i×c (response; λ_i = Lagrange coefficient)
Broadcasts z_i Aggregation: z = Σ z_i (aggregate response) Signature =
(R, z) – standard 64-byte Schnorr signature Verification (any party):
ASSERT z×G == R + c×aggregatePubKey
```

Appendix B — RailNamespace Registry (Current, v1.0)

Namespace	Status	Hash Function	Sig Scheme	Finality	Req. Confs	Registered Since
btc:mainnet	ACTIVE	SHA256d	Schnorr/ECDSA A secp256k1	Probabilistic	6	URIB v1.0
btc:testnet4	ACTIVE	SHA256d	Schnorr/ECDSA A secp256k1	Probabilistic	6	URIB v1.0
xrp:mainnet	ACTIVE	SHA512/256	ECDSA secp256k1 / Ed25519	Deterministic	1	URIB v1.0
xrp:testnet	ACTIVE	SHA512/256	ECDSA secp256k1 / Ed25519	Deterministic	1	URIB v1.0
iso20022:sepa	ACTIVE	SHA256 (c14n XML)	RSA-PSS / ECDSA	Instant	1	URIB v1.0
iso20022:swift	ACTIVE	SHA256 (c14n XML)	RSA-PSS	Deterministic	1	URIB v1.0

Namespace	Status	Hash Function	Sig Scheme	Finality	Req. Confs.	Registered Since
iso20022:fednow	ACTIVE	SHA256 (c14n XML)	RSA-PSS	Instant	1	URIB v1.0
evm:1:mainnet	ACTIVE	Keccak256	ECDSA secp256k1	Probabilistic	12	URIB v1.0
evm:137:polygon	ACTIVE	Keccak256	ECDSA secp256k1	Probabilistic (L2)	L1 state root	URIB v1.0
evm:42161:arbitrum	ACTIVE	Keccak256	ECDSA secp256k1	Probabilistic (L2)	L1 state root	URIB v1.0
evm:10:optimism	ACTIVE	Keccak256	ECDSA secp256k1	Probabilistic (L2)	L1 state root	URIB v1.0
evm:8453:base	ACTIVE	Keccak256	ECDSA secp256k1	Probabilistic (L2)	L1 state root	URIB v1.0
stellar:mainnet	ACTIVE	SHA256 (XDR)	Ed25519	Deterministic	1	URIB v1.0
stellar:testnet	ACTIVE	SHA256 (XDR)	Ed25519	Deterministic	1	URIB v1.0
cbdc:eur:ecb	ACTIVE	SHA256	ECDSA secp256k1	Instant	1	URIB v1.0
cbdc:usd:fed	ACTIVE	SHA256	ECDSA secp256k1	Instant	1	URIB v1.0
cbdc:multi:mbridge	ACTIVE	SHA256 / Keccak256	ECDSA secp256k1	Deterministic	1	URIB v1.0

Appendix C — ICT Derivation Reference

PSUEDOCODE: ICT DERIVATION PER RAIL (WITH TEST VECTORS)

```
// — Bitcoin P2WPKH —————
input_pubkey      =
0x0279be667ef9dcbbac55a06295ce870b07029bfcd8b2dce28d959f2815b16f81798
canonicalPubkeyHash = keccak256(input_pubkey) = 0x... (32
bytes) ICT(version=0) = keccak256(abi.encodePacked("btc:mainnet",
canonicalPubkeyHash, uint32(0)))
0x[expected_ICT_BTC_P2WPKH] // — XRP Classic Address
————— input_accountId =
0x5e7b0020fcd8a02e6a6f9a6e5e6a1fc3a39e1e5e // RIPEMD160(SHA256(pubkey))
canonicalPubkeyHash = keccak256(input_accountId) ICT(version=0) =
keccak256(abi.encodePacked("xrp:mainnet", canonicalPubkeyHash, uint32(0)))
// — ISO 20022 with LEI —————
bic      = UTF8_encode("DEUTDEDB") // 8 bytes iban =
UTF8_encode("DE89370400440532013000") // 22 bytes lei =
UTF8_encode("529900T8BM49AURSD055") // 20 bytes (ISO 17442)
canonicalPubkeyHash = keccak256(bic || iban || lei) ICT(version=0) =
keccak256(abi.encodePacked("iso20022:sepa", canonicalPubkeyHash, uint32(0)))
// — EVM EOA —————
// Standard secp256k1 uncompressed pubkey (65 bytes) uncompressed_pubkey =
0x04... (65 bytes) canonicalPubkeyHash = keccak256(uncompressed_pubkey)
ICT(version=0) = keccak256(abi.encodePacked("evm:1:mainnet",
canonicalPubkeyHash, uint32(0))) // Note: EVM checksummed address is NOT
used directly; always derive from pubkey. // — Stellar G-Address
————— ed25519_pubkey =
0xe3d0f8... (32 bytes, x-coordinate only) canonicalPubkeyHash =
keccak256(ed25519_pubkey) ICT(version=0) =
keccak256(abi.encodePacked("stellar:mainnet", canonicalPubkeyHash,
uint32(0))) // — Stellar Muxed M-Address child ICT
————— muxed_sub_id = uint64(42) subIdHash
= keccak256(abi.encodePacked(ed25519_pubkey, muxed_sub_id))
childICT(version=0) = keccak256(abi.encodePacked("stellar:mainnet",
subIdHash, uint32(0))) // — CBDC with RIA
————— cbdc_pubkey =
0x... (platform-specific pubkey bytes) riaHash =
keccak256(abi.encode(kycSubjectId, complianceTier, expiryTimestamp, sig))
canonicalPubkeyHash = keccak256(cbdc_pubkey || riaHash) ICT(version=0) =
keccak256(abi.encodePacked("cbdc:eur:ecb", canonicalPubkeyHash, uint32(0)))
```

Appendix D — Error Code Reference

Error Code	Category	Description	Recovery Action	Severity
E-MAP-001	Mapping	Unknown or unregistered rail namespace	Register namespace; retry	REJECT
E-MAP-002	Mapping	Amount overflow or zero-amount	Fix decimal conversion; use 512-bit intermediate	REJECT
E-MAP-003	Mapping	Identity not resolvable; no provisional ICT possible	Register identity in ICL	REJECT
E-MAP-004	Mapping	Invalid obligation type classification	Review CST-M-003 decision tree	REJECT
E-MAP-005	Mapping	Thread chain cycle detected	Inspect parentThreadId chain; remove circular reference	REJECT
E-MAP-006	Mapping	Semantic payload ABI encoding failure	Verify ABI library; check field types	REJECT
E-MAP-007	Mapping	Finality timeout — staleness threshold exceeded	Check rail node connectivity; restart finality watcher	WARN
E-MAP-008	Mapping	Epoch mismatch — epochAnchor not OPEN	Re-fetch epoch anchor; resubmit	REJECT
E-MAP-009	Mapping	CST schema violation — mandatory field missing	Review CST schema; fix field construction	REJECT
E-MAP-010	Mapping	Duplicate threadId — already in registry	Investigate threadId computation; check nativeTxId uniqueness	REJECT + ALERT
E-MAP-011	Mapping	FX rate stale — oracle commitment too old	Wait for next epoch; do not normalise with stale rate	REJECT
E-MAP-012	Mapping	Commitment digest mismatch — payload modified after digest	Recompute commitmentDigest; do not mutate payload post-digest	REJECT
E-INV-U-001	Invariant	Conservation violated — debits $\neq$ credits + fees	Fix amount computation; audit adapter	REJECT
E-INV-U-002	Invariant	Identity binding missing — ICT not verifiably bound	Register binding proof in ICL	REJECT
E-INV-U-003	Invariant	Epoch anchor mismatch — not OPEN epoch	Use current epoch anchor; resubmit	REJECT

Error Code	Category	Description	Recovery Action	Severity
E-INV-U-004	Invariant	Commitment retraction attempted — CRC is irrevocable	Invoke dispute resolution protocol	REJECT + ALERT
E-INV-U-005	Invariant	Finality monotonicity violated — reorg detected	Suspend affected threads; monitor reorg resolution	WARN → HALT
E-INV-U-006	Invariant	Duplicate threadId	Investigate adapter ID generation	REJECT + ALERT
E-INV-U-007	Invariant	C_AMT precision violation — synthetic value created	Audit adapter amount code; suspend adapter; re-certify	REJECT + SUSPEND
E-INV-U-008	Invariant	Semantic payload integrity failure — hash mismatch	Recompute payload hash; do not modify payload after digest	REJECT
E-INV-CBDC-001	Invariant / CBDC	KYC/AML attestation expired	Refresh attestation from national KYC registry	REJECT
E-INV-CBDC-002	Invariant / CBDC	Compliance tier limit exceeded	Verify tier assignment; split transaction if needed	REJECT
E-INV-CBDC-003	Invariant / CBDC	Unauthorised MINT obligation	Obtain governance authorisation; resubmit with auth hash	HALT
E-SEP-001	SEP	No threads found for obligation node	Verify CST Engine registration; re-submit missing threads	REJECT
E-SEP-002	SEP	Thread epoch anchor mismatch across SEP contributors	Verify all threads are in the same epoch	REJECT
E-SEP-003	SEP	SEP total value does not match obligation required value	Verify all contributing thread C_AMTs sum correctly	REJECT
E-ICT-001	ICT / ICL	ICT not found in registry	Register identity; resubmit	REJECT
E-ICT-002	ICT / ICL	ICT version EXPIRED — key rotation migration window closed	Update to current ICT version; re-register binding	REJECT
E-ICT-003	ICT / ICL	ICT status SUSPENDED — identity under dispute	Contact URIB governance; await dispute resolution	REJECT + ALERT
E-ICT-004	ICT / ICL	Binding proof invalid — signature does not verify	Re-generate binding proof with correct key; resubmit	REJECT
E-ICT-005	ICT / ICL	Provisional ICT expiry — binding not confirmed in 2 epochs	Submit confirmed binding proof; resubmit CST	REJECT

Appendix E — Changelog

Version	Date	Author	Summary of Changes
v1.0	July 2026	URIB Core Engineering Team	Initial release. Defines complete Rail Adapter Interface (RAI) specification, CanonicalSemanticThread (CST) schema and mapping rules (CST-M-001 through CST-M-006), eight Universal Rail Invariants (INV-U-001 through INV-U-008), Settlement Equivalence Proof (SEP) structure and generation algorithm, Identity Continuity Token (ICT) architecture and per-rail derivation templates (BTC, XRP, ISO20022, CBDC, EVM, Stellar), per-rail adapter specifications for all six initial rails, five worked examples covering major cross-rail scenarios, Bridge Orchestrator epoch lifecycle and Obligation Graph model, Security threat model and cryptographic agility framework, and Deployment and operations runbook including full configuration reference and error code table (E-MAP-001 through E-ICT-005).